# Self-interpretation in PTS-rk: a rank obstruction and an indexed approach

October 21, 2026

**Abstract**

The full original annotated PTS-rk admits a dependent indexed PHOAS self-interpreter. The Rocq construction gives a finite numerical allocation plan for every original typing derivation in a well-formed context, normal represented type and term programs, and a normal originally typed interpreter whose application reduces to the literal unreduced source term. The represented type is an ordinary object-language argument, not an external source-type index. A faithful raw HOAS tag preserves annotated syntax independently of conversion repairs; equality or beta conversion of normal quoted programs implies literal source equality at a fixed source scope, even across different signatures and derivations. Open terms are covered by literal dependent closure and reopening.

The construction combines a finite rank-indexed Tarski interface with independently enumerated and checked equality-only signature layers. Every repair has independently typed endpoints and an actual normal identity realizer. Structural quotation follows all six original typing rules, including dependent application and conversion, without recursively quoting synthesized validity derivations. The interpreter runs the represented PHOAS program; it does not project a precomputed semantic source value. No PTS rule, primitive, eta conversion or cumulativity is added.

The indices include a finite plan of telescope-length, syntax-node and literal bounds as well as ranks. Coverage by a rank-only family and a practical quotation-cost bound are not established. Fixed-rank elementary normalization and non-uniform elementary representability motivated this investigation, but the construction does not assume the manuscript-level elementary compiler. Restricted rank, literal-generation and shared-index obstructions remain valid; they do not contradict this sufficiently indexed positive result.

## 1 Why the complexity bounds suggest an indexed interpreter

There is positive computational evidence for a non-uniform self-interpreter. The checked normalization development covers the **full original annotated PTS**, not just a simply typed sublanguage. For a well-typed $d$-bounded term $t$ in a well-formed context, $d$ bottom-up greedy passes produce its normal form at the same original type. Writing $N$ for its node count and $\mathrm{tower}_0(x) = x$, $\mathrm{tower}_{h+1}(x) = 2^{\mathrm{tower}_h(x)}$, the proved operation-cost bound is

$$\mathrm{cost}(\mathsf{normalize}_d(t)) \leq \mathrm{tower}_{2d+1}(N + 5d + 2).$$

Thus normalization is elementary for each fixed external $d$. Moreover, every **closed, originally well-typed** term has the computable sufficient bound

$$d(t) = 1 + \mathsf{maxRankLiteral}(t) + \mathsf{nodes}(t).$$

For the documented encoding size $s$, this bound is at most $2s + 1$ and its traversal costs at most $7s$. These are the public Rocq results `fixed_rank_normalization` and

`rank_bound_in_linear_time`. The uniform automatic normalizer has an input-dependent tower-height bound; this does not provide a uniform fixed-height elementary bound on all inputs. The numerical rank bound here is a bound on redex heads, not merely the rank of the final result type. The open-context extension below computes a sufficient bound from both the term and its context declarations; it requires no normalization-success oracle.

The manuscript's computational-model section, theorem label `non-uniform-rep`, supplies the other half of the proposed argument: for each elementary function $f$ there is a rank cost $c_f$ and, at each base rank $j$, a numeral realizer

$$F_j : \mathcal{N}_{j+c_f} \to \cdots \to \mathcal{N}_{j+c_f} \to \mathcal{N}_j, \qquad \mathcal{N}_j = \Pi X : \mathcal{U}_j.(X \to X) \to X \to X,$$

computing $f$ on canonical Church numerals. The rank cost is computed from an explicit elementary program, using numeral-preserving rank lowerings and composition. It is not a rank inferred from arbitrary extensional black-box knowledge of $f$. The cited elementary substitution-basis theorem[1] gives the relevant arithmetic basis. A constructive compilation of the fixed-$d$ normalization algorithm would therefore yield an effective external rank budget depending on $d$, its elementary program and the chosen serialization; no numerical formula for that compiler budget is currently kernel-checked here.

**Proposition 1.1** (The complexity-to-self-reduction argument)**.** *Assume the manuscript's non-uniform elementary representability theorem and elementary encoding/decoding of annotated syntax as numbers. For each fixed $d$ and base rank $j$, there is an original PTS numeral program $S_{d,j} : \mathcal{N}_{j+c_d} \to \mathcal{N}_j$ which, on the numerical code of any originally typed $d$-bounded term, returns the numerical code of its full beta-normal form.*

*Proof.* On arbitrary numerical input, parse a raw annotated syntax tree, returning a fixed default for malformed input. Perform $d$ greedy passes and encode the result. For fixed $d$, the structural operations, serialization and fixed-height bounds give an elementary total function; no type-checker or unbounded search is needed in this algorithm. Apply non-uniform representability to that function. On the independently specified typed, $d$-bounded domain, the checked normalization theorem supplies correctness. The program is fixed for $d, j$, not chosen anew for each input term. □

This is a genuine route to an **indexed numeric self-reducer**. Its non-uniformity avoids the fixed-head rank obstruction. However, a number coding a normal form is not an inhabitant of the represented dependent type. A HOAS/PHOAS interpreter still needs actual typed adapters and reification, including context and conversion coherence. Further, normalization yields $\mathsf{nf}(t) =_\beta t$, not reduction back to the original unreduced $t$. These distinctions determine what the complexity argument proves and what remains to construct.

**Verification boundary.** Normalization and its stated operation costs are checked. The generic elementary-function compiler is not formalized; `latex/ROCQ_REFERENCES.md` explicitly records this boundary. The displayed modulus construction in the manuscript also needs the local callback correction $r\, f\, x$ in place of $r\, X\, f\, x$, because that record field is already specialized to $X$. We do not silently treat the uncorrected display as a checked inhabitant. The numerical proposition above is a mathematical conditional, not a new Rocq axiom or a completed internal HOAS interpreter.

[1] `https://arxiv.org/abs/2505.23787`

## 2 Question and source

The reference is Matt Brown and Jens Palsberg, *Breaking through the Normalization Barrier: A Self-Interpreter for F-omega*, POPL 2016, pp. 5–17, doi:10.1145/2837614.2837623[2]. The supplied PDF has filename `2914770.2837623.pdf`; the theorem labels below refer to its text, not to PDF page indices.

Their target is an **unquoter**: an external quotation operation assigns a typed, beta-normal representation $\ulcorner t \urcorner$ to each term $t$, and an object-language program recovers $t$ from this representation. The strong version requires reduction to the original term, $\mathsf{unquote} \ulcorner t \urcorner \longrightarrow_\beta^* t$, rather than merely conversion to it. Quotation must preserve syntax injectively, with the paper's precise treatment of equivalent type annotations. Returning the quotation of a normal form is a different operation, called a self-reducer.

Section 3.3, Theorems 3.4–3.5, gives the shallow construction. A neutral variable of type $\forall A : *.A \to A$ blocks each application; substituting the polymorphic identity unblocks it. Sections 4–7 replace this with a deeper representation using typed higher-order abstract syntax, extensional instantiation functions, and interpretations of types. Theorems 6.1–6.3 establish typing, normality, and injectivity up to equivalence of type annotations; Theorem 7.4 establishes strong recovery. Section 10 leaves dependent self-interpretation and self-representation without impredicativity as open directions.

These are not impossibility results from strong normalization alone. The paper specifically explains why a numerical diagonalization argument does not automatically apply to typed quotation. The obstruction below instead uses the exact PTS-rk product rule.

## 3 The original calculus and the rank invariant

All claims use the existing `PTSrk.Syntax.Term nat n`, including lambda annotations, `Typing.typ`, `Ranks.RankSorts`, and original beta reduction. There is no cumulativity, added primitive, eta conversion, or replacement typing judgment. Its sort and product rules are

$$\Gamma \vdash \mathcal{U}_i : \mathcal{U}_0, \qquad \frac{\Gamma \vdash A : \mathcal{U}_a \quad \Gamma, x : A \vdash B : \mathcal{U}_b}{\Gamma \vdash \Pi x : A.B : \mathcal{U}_{\max(a+1,b)}}.$$

The **type rank** of a typed term $t$ is $r$ when $\Gamma \vdash t : T$ and $\Gamma \vdash T : \mathcal{U}_r$. This is the rank of its type, not its largest sort literal or a bound on all redexes inside $t$. In well-formed contexts it exists, is unique, and is invariant under beta conversion of typed terms. These facts are already proved in `NormalizationComplexity/RedexRanks.v`.

**Lemma 3.1** (Application rank)**.** *If $\Gamma$ is well formed and $f\,u$ is typed, then* $\mathrm{rk}_\Gamma(f\,u) \le \mathrm{rk}_\Gamma(f)$.

*Proof.* Invert the typing of the application to obtain $f : \Pi x : A.B$ and $u : A$. If $A : \mathcal{U}_a$ and $B : \mathcal{U}_b$ under the binder, the type of $f$ has rank $\max(a+1, b)$. Substitution preserves $B : \mathcal{U}_b$, so $B[u/x]$ has rank $b$. Uniqueness of type rank removes any dependence on the chosen derivation or intervening conversion rules. □

Consequently, for any well-typed finite application spine,

$$\mathrm{rk}(E\,a_1 \cdots a_m) \le \mathrm{rk}(E).$$

[2] `https://doi.org/10.1145/2837614.2837623`

Arguments may be types, representations, or other terms. Their number and their construction may depend arbitrarily on the represented program. This bound already holds without a computable quotation operation.

**Theorem 3.2** (No universal fixed-head evaluator)**.** *There is no closed typed PTS-rk term $E$ such that every closed typed term $t$ is beta-convertible to a well-typed finite application spine headed by $E$.*

*Proof.* Define closed types and inhabitants by

$$A_0 = \mathcal{U}_0, \quad A_{r+1} = \Pi x : A_r.\mathcal{U}_0, \qquad v_0 = \mathcal{U}_0, \quad v_{r+1} = \lambda x : A_r.\mathcal{U}_0.$$

The original rules derive $A_r : \mathcal{U}_r$ and $v_r : A_r$ for every $r$. Suppose $E$ has type rank $R$. An application spine of $E$ has rank at most $R$, so invariance under conversion prevents it from being convertible to $v_{R+1}$. This contradicts universality. □

The Rocq declaration is `RankObstruction.no_universal_fixed_head_evaluator`. Its specification allows arbitrary externally chosen arguments and arbitrary intermediate result types. It assumes neither an encoding nor normality nor injectivity, making the exclusion stronger than failure of one particular encoding.

This excludes a direct port of the actual fixed unquote programs in Sections 3.3 and 7, including implicit type applications. A fixed wrapper chosen once is another fixed $E$ and is excluded as well. Section 3.2 also permits implicit type *abstractions*. If interpreted as arbitrarily many term-dependent outer wrappers, these lie outside the application-spine theorem: external abstraction can raise rank. We do not claim to have excluded every liberal interpretation of that allowance, externally generated interpreter families, or a weaker observational semantics.

# 4 What the sort cycle does and does not provide

Every $\mathcal{U}_r$ has type $\mathcal{U}_0$, but this does not place every type in $\mathcal{U}_0$. For example $A \to A$ has rank $r+1$ when $A : \mathcal{U}_r$. Thus there is no single sort playing the role of the arrow-closed kind $*$ of System $F_\omega$. Replacing every occurrence of $*$ by $\mathcal{U}_0$ in the paper's code does not produce a typing derivation.

There is, however, an identity at every external rank:

$$\mathsf{I}_r = \Pi A : \mathcal{U}_r.\, A \to A, \qquad \mathsf{id}_r = \lambda A : \mathcal{U}_r.\lambda x : A.x, \qquad \mathsf{I}_r : \mathcal{U}_{r+1}, \quad \mathsf{id}_r : \mathsf{I}_r.$$

The numerical index $r$ is part of generated syntax. Rocq quantification over $r : \mathbb{N}$ produces a family of original PTS terms; it is not an internal PTS quantifier over ranks. In particular an object numeral cannot simply be substituted for the sort label in $\mathcal{U}_r$.

For a blocker rank $r$ and an output-type rank $s$, define

$$\begin{aligned} \mathsf{Code}_r(A) &= \mathsf{I}_r \to A, \\ \mathsf{D}_{r,s} &= \lambda A : \mathcal{U}_s.\lambda q : \mathsf{Code}_r(A).\, q\, \mathsf{id}_r, \\ T_{r,s} &= \Pi A : \mathcal{U}_s.\, \mathsf{Code}_r(A) \to A. \end{aligned}$$

The exact original judgments, proved in `RankedDecoder.v`, are

$$A : \mathcal{U}_s \implies \mathsf{Code}_r(A) : \mathcal{U}_{\max(r+2,s)},$$
$$\mathsf{D}_{r,s} : T_{r,s}, \qquad T_{r,s} : \mathcal{U}_{\max(r+3,s+1)}.$$

For any typed code $q : \mathsf{Code}_r(A)$, $\mathsf{D}_{r,s}\, A\, q \longrightarrow_\beta^* q\, \mathsf{id}_r$. The raw packet $p_r(t) = \lambda i : \mathsf{I}_r.t$ (with the necessary de Bruijn shift) satisfies $\mathsf{D}_{r,s}\, A\, p_r(t) \longrightarrow_\beta^* t$ in three beta steps. This is a typed retraction for every original typed term, including dependent terms. It is **not a quotation theorem**: $p_r(t)$ retains every source redex. The declaration `typed_ranked_retraction` deliberately makes only the retraction claim.

The theorem `decoder_type_not_in_result_universe` formalizes a failure of self-specialization. The decoder cannot be instantiated at its own type in its output universe: $T_{r,s}$ has rank at least $s + 1$, hence cannot also inhabit $\mathcal{U}_s$. Increasing an external index is therefore a real change of program, not the paper's self-application of one polymorphic interpreter.

## 5 HOAS replacement for the complete annotated syntax

Terms and types share the original syntax. The former first-order Church AST is now replaced by a higher-order encoding: source variables become target variables, and source binding becomes actual target lambda binding. Original product domains and lambda annotations remain explicit data. This represents every raw annotated source tree, not only well-typed ones.

Fix a carrier rank $r$ and an external source scope size $n$. Write $X^n \to X$ for $n$ curried arguments of type $X$, with $X^0 \to X = X$, and put

$$\begin{aligned} H(X) &= X \to (X \to X) \to X, \\ \mathsf{HOAS}_{r,n} &= \Pi X : \mathcal{U}_r.\, X \to (X \to X) \to (X \to X) \\ &\qquad \to H(X) \to H(X) \to (X \to X \to X) \to (X^n \to X). \end{aligned}$$

The six handlers are numeral zero $z$, numeral successor $s$, sort $u$, product $p$, lambda $l$, and application $a$. There is no variable constructor. For an environment $\rho$ assigning carrier terms to source variables, the executable fold is

$$\begin{aligned} F_\rho(\mathcal{U}_k) &= u\,(s^k z), & F_\rho(x) &= \rho(x), \\ F_\rho(\Pi x : A.B) &= p\,(F_\rho(A))\,(\lambda x : X.F_{\rho,x}(B)), \\ F_\rho(\lambda x : A.b) &= l\,(F_\rho(A))\,(\lambda x : X.F_{\rho,x}(b)), \\ F_\rho(f\,t) &= a\,(F_\rho(f))\,(F_\rho(t)). \end{aligned}$$

The binder clauses shift every handler and outer environment entry and map the newly bound source variable to the new target variable. This is HOAS inside the original PTS syntax, not a Rocq function hidden in an abstract syntax constructor. The closed quote $C_{r,n}(t)$ abstracts over $X, z, s, u, p, l, a$, then over the $n$ carrier variables needed to close its source environment. For example, at source scope zero,

$$\begin{aligned} C_{r,0}(\lambda x : \mathcal{U}_k.x) = {} & \lambda X : \mathcal{U}_r.\lambda z : X.\lambda s : X \to X. \\ & \lambda u : X \to X.\lambda p : H(X).\lambda l : H(X). \\ & \lambda a : X \to X \to X.\; l\,(u\,(s^k z))\,(\lambda x : X.x). \end{aligned}$$

**Proposition 5.1** (Normal, faithfully encoded raw HOAS)**.** *For every original* $t : \mathsf{Term}\ \mathbb{N}\ n$ *and every external* $r$*, the original rules derive* $\vdash C_{r,n}(t) : \mathsf{HOAS}_{r,n}$ *and* $\vdash \mathsf{HOAS}_{r,n} : \mathcal{U}_{r+3}$*. The quotation is fully beta-normal. At each fixed source scope* $n$*,*

$$C_{r,n}(t) =_\beta C_{s,n}(t') \quad \Longleftrightarrow \quad r = s \text{ and } t = t'$$

*as literal annotated source syntax.*

The binder handler $H(X)$ has rank $r + 2$, so accepting this handler raises the code type to rank $r + 3$. This is one higher than the replaced first-order representation. Additional carrier-variable arguments do not raise that bound. The existence endpoint in `ChurchSyntax` is `every_original_term_has_church_code`. The fidelity theorem is `ChurchSyntaxFaithful.quote_conversion_iff`. Normality includes the generated lambda annotations. Injectivity is proved by structural inversion under an injective variable environment, not inferred from evaluation correctness. It therefore retains distinctions between a reducible source annotation and its contractum.

Applying a quotation to its carrier and six handlers performs seven administrative beta steps:

$$C_{r,n}(t)\, X\, z\, s\, u\, p\, l\, a \longrightarrow_\beta^* \lambda x_1 : X. \cdots \lambda x_n : X. F_{\vec{x}}(t).$$

`ChurchSyntaxFold.run_fold` establishes this reduction, and `run_typing` derives the original typing of the actual application with result type $X^n \to X$. No subject expansion is used. With $n = 0$ the result is the unwrapped fold. Source sort labels remain numeral data; no lower bound on $r$ is imposed by their magnitude.

**Terminological boundary.** This is a PTS-typed encoding of raw syntax. Its carrier does not depend on the source term's type. In the terminology of the paper it is an *untyped source representation*, despite its actual target typing theorem. It is not the source-type-indexed construction discussed next. No claim is made that every inhabitant of its code type arises from source syntax, or that its arbitrary algebras perform typed unquotation. Instantiating its carrier by another code type requires $r = s + 3$, without cumulativity or an internal rank quantifier.

# 6 Which indices a raw-HOAS decoder must retain

There are three different quantities: a redex bound $d$, the exact rank $s$ of the result type, and the largest literal sort label $\ell$ in the source. The conservative bound $d(t)$ in the introduction includes literal labels. An arbitrary sufficient redex bound need not do so: every $\mathcal{U}_k$ is already normal, has redex bound zero, and has type $\mathcal{U}_0$, of rank zero, regardless of $k$.

The raw HOAS quote deliberately turns these labels into numeral data. In fact, `LiteralObstruction.quote_literal_exact` proves

$$\mathsf{maxRankLiteral}(C_{r,n}(t)) = r.$$

**Theorem 6.1** (Literal-generation obstruction)**.** *For any fixed closed raw term* $E$ *and carrier index* $r$*, if* $E\, C_{r,0}(\mathcal{U}_k) =_\beta \mathcal{U}_k$*, then*

$$k \le \max\{\mathsf{maxRankLiteral}(E), r\}.$$

*Consequently no such $E$ unquotes all normal sorts from these codes, even when the source redex bound and result-type rank are both zero.*

*Proof.* Original beta reduction never creates a larger literal sort label. Conversion to the normal sort $\mathcal{U}_k$ implies reduction to it by the common-reduct theorem. The applied code contains only the literal labels of $E$ and the single carrier label $r$. Taking $k$ one greater than their maximum gives a counterexample. No typing premise on $E$ is needed. □

The theorem `no_fixed_type_argument_unquoter` also permits the ordinary type argument $\mathcal{U}_0$ before the raw code. This argument supplies none of the missing $k$. Thus indexing an interpreter by **only** redex bound and result-type rank does not solve raw-code unquotation. A family whose generated syntax additionally depends on a literal ceiling $\ell$, or on the conservative bound which includes it, is not excluded. Neither is a representation carrying additional literal-bearing semantic data. The obstruction does not concern numeric code-to-code self-reduction: an output code can represent $k$ by data without generating the literal $\mathcal{U}_k$.

# 7 Full-PTS normalization connected to HOAS

The executable metalevel construction in `RankedNormalization` is

$$\mathsf{NQ}_{r,d}(t) = C_{r,n}(\mathsf{rounds}_d(t)).$$

For every original typed, $d$-bounded term in a well-formed context, the source of this quotation is its full normal form, reachable by original reduction and retaining its original type. The resulting closed HOAS code is normal and has type $\mathsf{HOAS}_{r,n}$. For two such sources in the same context, possibly with different sufficient budgets and types, the checked theorem gives

$$\mathsf{NQ}_{r,d}(t) =_\beta \mathsf{NQ}_{r',e}(u) \quad \Longleftrightarrow \quad r = r' \text{ and } t =_\beta u.$$

The automatic variant uses $d(t)$ and covers every closed original typed term without a supplied bound or normalization-success hypothesis. It identifies beta-convertible sources, unlike the raw quotation, which retains exact syntax. In particular, canonicalization is not being mistaken for syntax-level injectivity.

The normalization stage inherits the operation cost from the introduction. The additional checked output-size bound, for $L = \mathsf{maxRankLiteral}(t)$ and $N = \mathsf{nodes}(t)$, is

$$\mathsf{nodes}(\mathsf{NQ}_{r,d}(t)) \le 34 + 2n + (2L + 5)\,\mathrm{tower}_d(N).$$

The factor involving $L$ accounts for unary label data in this HOAS representation. This is an output-size theorem, not a charged runtime for HOAS construction or numeric serialization. The canonicalizer is an executable Rocq function on the original syntax; it is not yet a PTS term implementing the conditional numerical program from the introduction.

### 7.1 Open contexts, canonical types and hereditary substitution

The canonicalizer now covers **every original well-typed open term**. Let $L(\Gamma)$ and $S(\Gamma)$ be the maximum literal label and maximum node count among the declarations of $\Gamma$, with empty maxima zero. Define executable metalevel functions

$$\begin{aligned} d(\Gamma, t) &= 1 + \max\{L(\Gamma), \mathsf{maxRankLiteral}(t)\} + \max\{S(\Gamma), \mathsf{nodes}(t)\}, \\ N_\Gamma(t) &= \mathsf{rounds}_{d(\Gamma,t)}(t). \end{aligned}$$

`OpenCanonical.budget_sound` proves this bound sufficient from original well-formedness and typing. `nf_correct` proves full normality, reduction from $t$, original type preservation and success of the bounded normalizer. The function consumes syntax, not an erased typing proof or a supplied normalization certificate. Its correctness requires original well-typedness; arbitrary raw input need not be normalized by these finite rounds. The normalization-stage cost is the earlier tower bound at $d(\Gamma, t)$; no new runtime bound for context scanning or code construction is claimed.

For originally typed terms, possibly in different well-formed contexts of the same scope,

$$N_\Gamma(t) = N_\Delta(u) \quad \Longleftrightarrow \quad t =_\beta u.$$

This compares original scoped syntax, not denotations of variables in unrelated environments. In particular, changing a valid typing context or sufficient normalization budget cannot change a term's normal form.

Let $\sigma : \Gamma \Rightarrow \Delta$ be an **actual original typed substitution**: $\Delta \vdash \sigma(i) : \sigma(\Gamma!!i)$ for each $i$. Set

$$\overline{\sigma}(i) = N_\Delta(\sigma(i)), \qquad H_{\Delta,\sigma}(t) = N_\Delta(t[\sigma]).$$

**Theorem 7.1** (Full-PTS canonical substitution)**.** *For well-formed original contexts, $\Gamma \vdash t : T$ and original typed $\sigma : \Gamma \Rightarrow \Delta$, the normalized substitution $\overline{\sigma}$ is again originally typed, and*

$$\begin{aligned} N_\Delta(t[\sigma]) &= N_\Delta\big(N_\Gamma(t)[\overline{\sigma}]\big), \\ \Delta &\vdash H_{\Delta,\sigma}(t) : H_{\Delta,\sigma}(T). \end{aligned}$$

*For original typed $\tau : \Delta \Rightarrow \Theta$, composition gives*

$$H_{\Theta,\tau}\big(H_{\Delta,\sigma}(t)\big) = H_{\Theta,\rho}(t), \qquad \rho(i) = H_{\Theta,\tau}(\sigma(i)).$$

*Identity substitution returns $N_\Gamma(t)$.*

*Proof.* Reduction commutes with simultaneous substitution in both the term and its entries. Normalizing an entry also reduces its dependent lookup type; original validity, preservation and conversion establish `TypedSub` for the normalized entries. The substituted terms are originally typed and convertible, so normal-form uniqueness yields literal equality. Validity supplies the exact sort needed to normalize the result type too, without cumulativity. Composition follows from ordinary substitution composition and these normalization equations. □

`CanonicalSubstitution` proves these statements for the **full original calculus**, including dependent products, sorts, conversion and annotations. The outer normalization cannot be dropped. A checked regression uses

$$\Gamma = (f : \Pi X : \mathcal{U}_0.\mathcal{U}_0), \quad A = f\,\mathcal{U}_0, \quad \sigma(f) = \lambda X : \mathcal{U}_0.X.$$

The source type and substitution entry are both normal and originally typed, but $A[\sigma] = (\lambda X : \mathcal{U}_0.X)\,\mathcal{U}_0$ is a redex with normal form $\mathcal{U}_0$. Normalizing entries alone does not make ordinary substitution canonical.

Contexts can also be normalized without changing the term being typed:

$$\mathsf{NC}(\varnothing) = \varnothing, \qquad \mathsf{NC}(\Gamma, A) = \mathsf{NC}(\Gamma), N_\Gamma(A).$$

`CanonicalContexts` constructs original well-formedness and actual identity substitutions in **both directions**. For well-formed $\Gamma$,

$$\Gamma \vdash t : T \quad \Longleftrightarrow \quad \mathsf{NC}(\Gamma) \vdash t : T.$$

The terms $t, T$, including annotations, are literally unchanged. The declarations become normal; context normalization is idempotent and reachable by original context reduction. Computing $N(t)$ in either context gives the same result. Formation ranks come from the original `Type`-valued well-formedness witness, not elimination of an existential `Prop` proof.

We therefore obtain concrete **distinct** syntax and type-index maps:

$$C_{r,n}(t), \qquad J_{r,\Gamma}(A) = C_{r,n}(N_\Gamma(A)).$$

`CanonicalTypeIndex` proves that the former retains literal annotated syntax, whereas equality of the latter, on originally typed inputs at fixed $r$, is exactly original beta conversion. Canonical indices satisfy the hereditary substitution and composition equations after quotation. On the convertible types $(\lambda X : \mathcal{U}_0.X)\,\mathcal{U}_0$ and $\mathcal{U}_0$, faithful codes differ while canonical indices agree; the faithful code of the first is not even convertible to its own canonical index. These maps genuinely avoid the shared-map obstruction.

Both maps produce closed inhabitants of the **homogeneous HOAS code type**; $J(A)$ is not itself a source type at the universe of $A$. Hereditary substitution here acts on original syntax before quotation, not by running an object-language program on a code. This completes a full-PTS canonical context/index algebra, but not an internal operation $\mathsf{El}(J(A)) =_\beta A$ or a translation of every dependent family to abstract represented arguments. These remain object-language obligations, not consequences to assume from normalization. No general PHOAS impossibility follows.

## 7.2 A full dependent weak interpretation with faithful syntax

There is nevertheless an actual closed indexed PTS decoder for a weaker representation. Normalize both the source and its type **during external quotation**, and retain the original raw HOAS tag to preserve syntax. Put $C_r = \mathsf{HOAS}_{r,0}$ and, for $A : \mathcal{U}_s$, define

$$
\begin{aligned}
W_r(A) &= (A \to C_r \to A) \to A,\\
\mathsf{pack}_r(A,u,c) &= \lambda h : A \to C_r \to A.\, h\, u\, c,\\
\mathsf{select}_r(A) &= \lambda x : A.\lambda c : C_r.x,\\
\mathsf{WDec}_{r,s} &= \lambda A : \mathcal{U}_s.\lambda q : W_r(A).\, q\,(\mathsf{select}_r(A)).
\end{aligned}
$$

Its exact original typings are

$$
\begin{aligned}
W_r(A) &: \mathcal{U}_{\max(s+2,r+5)},\\
\mathsf{WDec}_{r,s} &: \Pi A : \mathcal{U}_s.\, W_r(A) \to A,\\
(\Pi A : \mathcal{U}_s.\, W_r(A) \to A) &: \mathcal{U}_{\max(s+3,r+6)}.
\end{aligned}
$$

For a closed original judgment $t : A$, let $\mathsf{nf}(v) = \mathsf{rounds}_{d(v)}(v)$ and define

$$
Q_r(t : A) = \mathsf{pack}_r(\mathsf{nf}(A), \mathsf{nf}(t), C_{r,0}(t)).
$$

**Theorem 7.2** (Full closed weak interpretation)**.** *Every original closed derivation $\vdash t : A$ has a sort witness $\vdash A : \mathcal{U}_s$. The computed $Q_r(t : A)$ is fully normal, has original type $W_r(\mathsf{nf}(A))$, and satisfies*

$$
\mathsf{WDec}_{r,s}\, \mathsf{nf}(A)\, Q_r(t : A) \longrightarrow_\beta^* \mathsf{nf}(t) =_\beta t.
$$

*The applied decoder has original type $A$. Moreover, conversion between two such normal packages, even with different supplied source types, implies equality of their carrier indices and literal equality of their annotated source terms.*

*Proof.* Original type validity supplies $s$; normalization and preservation give $\mathsf{nf}(A) : \mathcal{U}_s$ and $\mathsf{nf}(t) : A$. Original conversion then types the payload at $\mathsf{nf}(A)$. Both payload and type are normal; the raw HOAS tag is normal independently of source reducibility. The package body is headed by its bound variable, so the whole quotation, including its annotation, is normal. The decoder performs five administrative beta steps to the payload. Its result can be converted back to type $A$. Conversion between normal packages implies their literal equality; extracting their tags and applying raw-HOAS fidelity recovers the exact source syntax. □

`WeakFullInterpretation.full_closed_weak_interpretation` proves coverage from original closed typing alone. `quote_conversion_injective` proves the separate fidelity claim. The supplied source type or derivation is not asserted injectively recoverable: only the annotated term is tagged. The quote is a total external syntax computation; its normality theorem has the original typing premise. The decoder's $s$ is an **exact** sort rank, obtained propositionally by type validity, not a larger bound used via nonexistent cumulativity.

This construction includes dependent products, sort terms, reducible annotations and all other original closed typed syntax. It requires no elementary-representability assumption. But its computation occurs in the external quoter, and its decoder ignores the HOAS tag. It is therefore a normal, faithful **weak semantic-payload interpretation**, not an internal interpreter of pure HOAS and not Brown–Palsberg strong unquotation of the unreduced source. The literal obstruction is avoided because the normal payload can itself contain the source's literal labels. The computational challenge left by the introduction is to move normalization and typed reification into the object program instead of supplying this payload.

# 8 Independent Brown–Palsberg PHOAS investigation

The paper's Figure 3 abstracts a type interpretation $F : * \to *$. For the arrow fragment its pretranslation uses $\widehat{A \to B} = F\widehat{A} \to F\widehat{B}$, while the representation of a term of type $A$ has carrier $F\widehat{A}$. Its abstraction and application handlers therefore involve $F(FA \to FB)$. Unquotation specializes this same $F$ to the type identity.

**Theorem 8.1** (Obstruction to the literal single-family interface)**.** *Let $F : \Pi A : \mathcal{U}_a.\mathcal{U}_b$. If $A, B : \mathcal{U}_a$ and $F(FA \to FB)$ has any original PTS type, then $a = b + 1$. If the same family type admits the identity $\lambda A : \mathcal{U}_a.A$, then $a = b$. Thus these requirements cannot hold simultaneously, even when different input and output ranks are allowed.*

*Proof.* Application inversion and uniqueness of typing force the argument $FA \to FB$ to inhabit $\mathcal{U}_a$. Both $FA$ and $FB$ inhabit $\mathcal{U}_b$, so the original product rule places that arrow in $\mathcal{U}_{b+1}$. Uniqueness of sort ranks gives $a = b + 1$; conversion cannot change it. The literal identity already has type $\Pi A : \mathcal{U}_a.\mathcal{U}_a$. Uniqueness of typing followed by product and sort conversion inversion therefore gives $a = b$. The two equations contradict each other. □

`BrownPalsberg.no_single_family_identity_interface` proves this statement from actual original typing judgments, including arbitrary alternative types used in application derivations. `no_literal_interface` specializes the arguments to the explicitly constructed closed type $A_a : \mathcal{U}_a$, so the obstruction is not a vacuous lack of source types. This is independent of the fixed-head evaluator theorem. It excludes the literal common-family interface, not every possible PHOAS encoding.

## 8.1 A rank-indexed repair of the type interpretation

For the independent simply typed source, replace the one family by external parameters $F_i : \Pi A : \mathcal{U}_i.\mathcal{U}_i$ and a ground carrier $o : \mathcal{U}_0$. Define

$$\begin{aligned} \widehat{o} &= o, \\ \widehat{A \to B} &= F_a\widehat{A} \to F_b\widehat{B}, \quad a = \mathrm{ord}(A), \quad b = \mathrm{ord}(B), \\ \mathcal{I}_F(A) &= F_{\mathrm{ord}(A)}\widehat{A}. \end{aligned}$$

Both $\widehat{A}$ and $\mathcal{I}_F(A)$ inhabit precisely $\mathcal{U}_{\mathrm{ord}(A)}$. The executable `pretype` and `interpreted_type` have original typing proofs, strong recovery to the ordinary source type code under $F_i = \lambda A : \mathcal{U}_i.A$, and proved finite dependence on indices at most $\mathrm{ord}(A)$. These are syntactic functions generating PTS terms; the index $i$ is external, not a new PTS sort variable.

With $c = \max(a + 1, b)$, the corresponding polymorphic handlers are

$$\begin{aligned} \mathsf{Abs}_{a,b} &= \Pi A : \mathcal{U}_a.\Pi B : \mathcal{U}_b.\, (F_aA \to F_bB) \to F_c(F_aA \to F_bB), \\ \mathsf{App}_{a,b} &= \Pi A : \mathcal{U}_a.\Pi B : \mathcal{U}_b.\, F_c(F_aA \to F_bB) \to F_aA \to F_bB. \end{aligned}$$

Both handler types inhabit $\mathcal{U}_{c+1}$; `indexed_handler_type_sorts` proves their formation. When the families are identities, the actual original lambda terms

$$\begin{aligned} \mathsf{abs}^{\mathrm{id}}_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : \mathcal{U}_b.\lambda f : A \to B.f, \\ \mathsf{app}^{\mathrm{id}}_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : \mathcal{U}_b.\lambda f : A \to B.\lambda x : A.f\, x \end{aligned}$$

inhabit these exact interpreted handler types by conversion.

`indexed_identity_handlers_typing` proves their original typing. The two strong beta equations are `indexed_abs_recovers` and `indexed_app_recovers`. Each is a closed original term at its external pair $(a, b)$. This genuinely repairs the local single-family rank conflict: the outer arrow uses $F_c$, not $F_a$ or a universal $F$. It does not by itself supply a closed quotation package or a dependent conversion theorem.

### 8.2 Source-type-indexed HOAS term prequotation

The new `TypedHOAS` construction uses the independent computational source derivations described below. Unlike the raw-AST representation, its result type records the source type. Write $I(A) = \mathcal{I}_F(A)$ and suppose specialized operations are supplied with types

$$\mathsf{abs}_{A,B} : (I(A) \to I(B)) \to I(A \to B), \qquad \mathsf{app}_{A,B} : I(A \to B) \to I(A) \to I(B).$$

The executable derivation-directed prequotation is

$$\begin{aligned} q(x) &= \rho(x), \\ q(\lambda x : |A|.t) &= \mathsf{abs}_{A,B}(\lambda x : I(A).q(t)), \\ q(f\, u) &= \mathsf{app}_{A,B}\, q(f)\, q(u), \end{aligned}$$

where the source derivation supplies $A, B$ and all binder lifts are explicit in the original scoped syntax. The theorem `prequote_typed` derives $q(t) : I(C)$ from independent source evidence $t : C$, a correctly interpreted source-variable environment, and the stated family and operation typings. The premises type the algebra, not its recursively constructed output.

The bounded variant uses the computable budget $K(D) = \max(\mathrm{ord}(C), b(D))$ for a derivation $D : t : C$, where $b(D)$ is the application budget defined below. The theorem `prequote_bounded_typed` requires family laws only at $i \le K(D)$ and case laws only when $\mathrm{ord}(A \to B) \le K(D)$. Including the result order matters: a high-order identity lambda can have application budget zero. Bounded case laws still quantify over source types, not merely a finite list of monomorphic variables. The finite implementation below specializes the rank-pair-polymorphic handlers above.

Specializing the families to the type identities and the operations to the actual closed Brown–Palsberg handlers gives strong recovery of the original annotated term (`identity_interpretation_recovers`). This is also related to actual replacement of already generated syntax: `prequote_substitution` proves that simultaneous substitution commutes with prequotation, provided it sends the embedded source variables to the stated new variables. The theorem `substitution_identity_recovers` then reduces that substituted prequote to the source under the new renaming, assuming the substitution actually supplies the specified identity families and handlers. It does not assume that the quotation itself recovers, nor assert existence of the missing finite substitution telescope. Annotation recovery is part of the original reduction proof.

`TypedHOASAlgebra` proves original typing of the actual two-application specializations of the closed identity handlers. Its theorem `typed_identity_interpretation` combines original typing at the ordinary source type with strong recovery, for every independent source derivation in an originally typed source environment. This concrete identity interpretation is not asserted normal: normal prequotation and subsequent identity interpretation are different phases.

In `TypedHOASFaithful`, neutral variable families and handlers give full annotation normality (`prequote_normal`). A structural eraser separately strips interpreted type applications and the two constructor patterns; `erase_prequote` yields exactly rename$(\rho, t)$. Consequently `prequote_injective` recovers literal annotated source syntax when $\rho$ is injective, even when the two derivations, result types and handler palettes differ. `prequote_conversion_injective` strengthens this to comparison by original beta conversion of the neutral prequotes. This is not proof irrelevance for derivation evidence.

These earlier results concern parameterized prequotation. The generic typing theorem and the variable-palette normality theorem cannot be combined by silently assuming infinitely many differently typed variables in one finite context. The following construction discharges that obligation with an actual finite context and actual typed parameter substitutions. Nor is every neutral interpretation family impossible in a finite context: one $h : \mathsf{I}_0$ already gives $F_r = h\,\mathcal{U}_r$, because $\mathcal{U}_r : \mathcal{U}_0$. The construction below uses separate variable families, not this alternative.

### 8.3 Closing the finite rank-indexed PHOAS representation

For each external $k$, bind $o : \mathcal{U}_0$, the $k{+}2$ families $F_0, \ldots, F_{k+1}$, and two polymorphic handlers $\mathsf{Abs}_{a,b}, \mathsf{App}_{a,b}$ for each $0 \le a, b \le k$. The handler types are the indexed types displayed above. This deliberately includes some unused pairs. Their arrow rank can be $k+1$, which explains the extra family $F_{k+1}$. The finite parameter context $\Delta_k$ has

$$1 + (k+2) + 2(k+1)^2$$

binders. Its original well-formedness, all annotation normality, and the actual ground, family and handler lookup typings are proved. It depends on $k$, not on the number of source type shapes or the syntax of a particular program. There are infinitely many simple types at fixed order; the handlers cover them by ordinary PTS type application, not by adding one variable for each type.

Specialize a handler at the two pretypes:

$$\mathsf{abs}_{A,B} = \mathsf{Abs}_{a,b}\,\widehat{A}\,\widehat{B}, \qquad \mathsf{app}_{A,B} = \mathsf{App}_{a,b}\,\widehat{A}\,\widehat{B}, \qquad a = \mathrm{ord}(A),\ b = \mathrm{ord}(B).$$

These applied handlers remain neutral. `PHOASAlgebra` proves their actual original typing and that of the resulting derivation-directed prequotation. `PHOASFaithful` accounts for the two extra type arguments in a new eraser; it proves full normality and literal annotated-source injectivity, also when comparing quotes by beta conversion. No disjointness of handler and family variable names is assumed by the generic eraser theorem.

For a term-closed independent source derivation $D : t : A$ whose sole free parameter is $o$, and $K(D) \le k$, put

$$\begin{aligned} \mathsf{PCode}_{k,A} &= \Pi\Delta_k.\,\mathcal{I}_F(A), \\ \mathsf{PQuote}_k(D) &= \lambda\Delta_k.\,q_F(D). \end{aligned}$$

Here $\Pi\Delta_k$ and $\lambda\Delta_k$ are notation for finite lists of **original** products and annotated lambdas; they are not added syntax. `PHOASTelescope` implements their closure and actual sequential application. Its application-typing theorem uses a typed substitution for the supplied arguments, not subject expansion from a known reduct. `PHOASPackage` constructs the concrete

context and code. The computed quote is closed, normal, and originally typed at $\mathsf{PCode}_{k,A}$. At fixed $k$, conversion of quotes implies literal equality of their annotated sources, even for different source derivations and result types.

`PHOASRecovery` proves that replacing the used families by their type identities and the handlers by their actual closed identity interpretations reduces the prequote to the original source. Its hypotheses concern only the ranks and rank pairs within $K(D)$; no correctness property is required of fallback entries outside the finite palette. This is a theorem about actual replacement of generated syntax, not about separately generating a second term with different parameters.

The parameter substitution is itself realized by an actual closed decoder. Writing $\boldsymbol{\iota}_k$ for the finite list of type identities and identity Abs/App handlers in the telescope's binder order, put

$$\mathsf{PDec}_{k,A} = \lambda q : \mathsf{PCode}_{k,A}.\lambda o : \mathcal{U}_0.q\, o\, \boldsymbol{\iota}_k.$$

**Theorem 8.2** (Closed finite PHOAS unquotation)**.** *For every independent term-closed Church-STLC derivation $D : t : A$ and $k \geq K(D)$, the actual original closed terms satisfy*

$$\begin{aligned} \mathsf{PQuote}_k(D) &: \mathsf{PCode}_{k,A}, \\ \mathsf{PDec}_{k,A} &: \mathsf{PCode}_{k,A} \to (\Pi o : \mathcal{U}_0.|A|), \\ \mathsf{PDec}_{k,A}\, \mathsf{PQuote}_k(D) &\longrightarrow_\beta^* \lambda o : \mathcal{U}_0.t. \end{aligned}$$

*The quote is normal and injective on original annotated source syntax at fixed $k$, even under comparison by target beta conversion.*

`PHOASDecoder.closed_phoas_interpretation` proves the typing and strong recovery claims; the separate package theorem proves fidelity. The ground abstraction is inside the decoder, not an external operation appended to evaluation. All finite parameter-typing and substitution premises are discharged by the constructed context and identity arguments. Code-type formation is proved independently of source inhabitation. The current sort theorems provide a universe witness, not a separately verified closed-form rank formula.

This decoder is indexed by **both $k$ and the full simple type** $A$, not by $k$ and $\mathrm{ord}(A)$ alone. At fixed $(k, A)$ it is the same raw PTS program for every source derivation within budget, independent of program size; it is not chosen per source term. Internalizing the type interpretation as an additional argument, to remove the full-$A$ external index, is not provided by this earlier decoder. The later dependent program decoder supplies an ordinary represented-type argument instead.

The executable regression quotes $\lambda x : o.(\lambda y : o.y)x$ with computed budget one. Its actual closed PHOAS package computes to a normal term, while its decoder strongly recovers the original still-reducible source, including the outer ground abstraction. No normal form or semantic value is precomputed and stored in this quotation. This completes the finite PHOAS construction for the independent STLC source, not for all original dependent PTS syntax.

This is a Church-style, algebra-parametric typed HOAS construction inside PTS-rk. The term PHOAS here emphasizes that the representation abstracts its interpretation families and operations. It should not be confused with adding a native Rocq inductive datatype

and calling that an internal PTS representation. In Chlipala's PHOAS[3], the variable family is likewise an explicit parameter, but that work's metatheoretic treatment is not a theorem about all inhabitants of these PTS code types. Our proofs concern the actual quoted outputs; they assume no parametricity axiom or no-junk theorem for arbitrary code inhabitants.

### 8.4 Genuinely dependent indexed handlers

The local rank-indexed repair is not limited to constant codomains. Let $F_i : \mathcal{U}_i \to \mathcal{U}_i$, $A : \mathcal{U}_a$ and $B : (F_a A) \to \mathcal{U}_b$, and set $c = \max(a+1, b)$ and

$$D_F(A, B) = \Pi x : F_a A.\, F_b(B\, x).$$

The actual original product rule gives $D_F(A, B) : \mathcal{U}_c$. Consequently the two dependent constructor interfaces are

$$\begin{aligned}\mathsf{DAbs}_{a,b} &= \Pi A : \mathcal{U}_a.\Pi B : (F_a A \to \mathcal{U}_b).\, D_F(A, B) \to F_c(D_F(A, B)),\\ \mathsf{DApp}_{a,b} &= \Pi A : \mathcal{U}_a.\Pi B : (F_a A \to \mathcal{U}_b).\, F_c(D_F(A, B)) \to D_F(A, B).\end{aligned}$$

Both inhabit exactly $\mathcal{U}_{c+1}$. In particular, the codomain of an applied constructor $h_{\mathrm{app}} : \mathsf{DApp}_{a,b}$ really depends on the represented argument:

$$h_{\mathrm{app}}\, A\, B\, q_f\, q_u : F_b(B\, q_u).$$

At identity families, the closed original handlers

$$\begin{aligned}\mathsf{dabs}_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : A \to \mathcal{U}_b.\lambda f : (\Pi x : A.Bx).f,\\ \mathsf{dapp}_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : A \to \mathcal{U}_b.\lambda f : (\Pi x : A.Bx).\lambda x : A.f\, x\end{aligned}$$

inhabit these exact interpreted interfaces by proved beta conversion. Their applications reduce to $f$ and $f\, u$, respectively. `DependentPHOAS` proves formation, operation typing, identity specialization and these reduction equations. It also derives the original type $B\, u$ of the actual applied identity App handler directly, before reducing it. No eta conversion is used.

These local handlers alone do not construct the supplied family $B : (F_a A) \to \mathcal{U}_b$ recursively from every source dependent family $B_0 : A \to \mathcal{U}_b$. The domains differ while $F_a$ is abstract. Treating this input as already translated would assume away the central coherence obligation.

For the candidate convention that interprets sort literals by themselves, a source type $T : \mathcal{U}_b$, regarded as a source term, has representation carrier $F_0(\mathcal{U}_b)$. Using that representation as an annotation requires a separate operation

$$\mathsf{El}_b : F_0(\mathcal{U}_b) \to \mathcal{U}_b.$$

Its type inhabits $\mathcal{U}_1$, and the type identity at rank $b$ is an actual inhabitant at identity families. Both facts and its beta equation are checked. Merely binding an abstract $\mathsf{El}_b$ does not give it the definitional conversion laws required by a dependent quoter.

[3] `https://adam.chlipala.net/papers/PhoasICFP08/`

### 8.5 A precise coherence obstruction, not a PHOAS impossibility

**Theorem 8.3** (Faithful normal syntax cannot also be a semantic type index)**.** *Let a fixed externally indexed domain contain the two closed original types*

$$T_1 = (\lambda X : \mathcal{U}_0.X)\,\mathcal{U}_0, \qquad T_0 = \mathcal{U}_0.$$

*No one map $Q$ on that domain can simultaneously have normal outputs, be injective on literal syntax, and preserve original beta conversion. The same holds if a separate conversion-preserving index map $J$ is required to satisfy $Q(T) =_\beta J(T)$ pointwise.*

*Proof.* Both types inhabit $\mathcal{U}_0$, and $T_1 \longrightarrow_\beta^* T_0$ while $T_1 \neq T_0$. Conversion preservation makes their normal images convertible, hence equal by original confluence. Injectivity gives the contradictory source equality. For the two-map variant, pointwise conversion transfers conversion preservation from $J$ to $Q$. □

`PHOASConversion` proves the statement at any target scope and external index. Its two witnesses already fit one independently specified domain: type rank zero, literal ceiling zero, at most five nodes, and sufficient redex bound six. Increasing those indices cannot repair the three simultaneous requirements once both witnesses occur at the same index. The argument does **not** force two genuinely different maps to agree. Canonical semantic indices plus faithful syntax codes, or another explicit coherence mechanism, lie outside the obstruction.

There is also a concrete source regression: the neutral PHOAS quotes of $\lambda x : o.(\lambda y : o.y)x$ and $\lambda x : o.x$ are provably not convertible, although the sources differ by one beta step. This is expected of faithful quotation, not a defect in its interpretation theorem. For dependent syntax, a quotation map therefore cannot simply be reused unchanged as the map interpreting type conversion. This is the precise obstruction established here; arbitrary indexed dependent PHOAS has not been ruled out.

## 9 Object-language Tarski codes and dependent transport

The canonical metalevel algebra suggests keeping semantic type indices separate from faithful syntax. A constructive next step is to make the necessary local transports **actual original PTS terms**. This section records the local operations used by the full construction later in the note.

### 9.1 Rank-indexed equality and two-way casts

For $A : \mathcal{U}_a$ and $u, v : A$, define the Leibniz transport type

$$\mathsf{Eq}_b(A, u, v) = \Pi P : (A \to \mathcal{U}_b).P\,u \to P\,v, \qquad \mathsf{refl}_b(A, u) = \lambda P : (A \to \mathcal{U}_b).\lambda z : P\,u.z.$$

`IndexedEquality` constructs these original annotated terms and proves

$$\mathsf{Eq}_b(A, u, v) : \mathcal{U}_{\max(a+2,b+1)}, \qquad \mathsf{refl}_b(A, u) : \mathsf{Eq}_b(A, u, u).$$

The explicit transport $e\,P\,z$ has type $P\,v$ when $z : P\,u$, and reflexivity transport reduces to the literal $z$. There are also closed polymorphic reflexivity programs, exact substitution laws and normality results. These are not new equality primitives.

Predicate ranks matter. The familiar impredicative proof of symmetry cannot simply instantiate $P$ with this equality type at the same level: the equality type itself has a larger sort. For the casts needed here, use $S, T : \mathcal{U}_c$ and a witness

$$e : \mathsf{Eq}_{c+1}(\mathcal{U}_c, S, T).$$

Its type has sort $\mathcal{U}_{c+2}$. Both arrow motives below have values of sort $\mathcal{U}_{c+1}$, so the following are well-typed:

$$\mathsf{cast}^{+}_{c,S}(e) = e\,(\lambda X : \mathcal{U}_c.S \to X)\,(\lambda x : S.x) : S \to T,$$
$$\mathsf{cast}^{-}_{c,S}(e) = e\,(\lambda X : \mathcal{U}_c.X \to S)\,(\lambda x : S.x) : T \to S.$$

`TarskiCasts.casts_typed` proves these application typings directly. For reflexivity evidence both functions reduce to $\lambda x : S.x$; applied casts therefore return the original argument, without eta. Abstract neutral evidence instead blocks computation. If the type, evidence and argument are normal and the evidence is not lambda-headed, the applied casts are normal. This is a **local** normality theorem; normality of every generated dependent annotation is not being assumed.

`ClosedTarskiCasts` closes these operations over $S$, $T$ and $e$. For each external $c$ the actual closed programs are normal, their types have sort $\mathcal{U}_{c+3}$, and their application typings and beta specialization are proved. Thus transport is an internal program component, not merely an external proof that two source types have the same normal form.

### 9.2 Dependent product and universe codes

For each needed external rank $a$, consider object parameters

$$C_a : \mathcal{U}_0, \qquad \mathsf{El}_a : C_a \to \mathcal{U}_a.$$

These are semantic type-code carriers, not the faithful homogeneous raw-HOAS carrier. An arbitrary inhabitant is not asserted to be generated syntax. For $c = \max(a+1, b)$, the product-code operation has type

$$p_{a,b} : \Pi A : C_a.(\mathsf{El}_a A \to C_b) \to C_c.$$

`TarskiProducts` proves that this operation type inhabits $\mathcal{U}_{a+2}$. For $A : C_a$ and $B : \mathsf{El}_a A \to C_b$, set

$$X = \Pi x : \mathsf{El}_a A.\mathsf{El}_b(Bx), \qquad Y = \mathsf{El}_c(p_{a,b}\,A\,B).$$

Both $X$ and $Y$ inhabit $\mathcal{U}_c$. An explicit object witness $e : \mathsf{Eq}_{c+1}(\mathcal{U}_c, X, Y)$ gives dependent operations

$$\mathsf{Abs}_e(f) = \mathsf{cast}^{+}_{c,X}(e)\,f : Y, \qquad \mathsf{App}_e(g,u) = \mathsf{cast}^{-}_{c,X}(e)\,g\,u : \mathsf{El}_b(Bu),$$

where $f : X$, $g : Y$ and $u : \mathsf{El}_a A$. The family $B$ is genuinely dependent. Original typing of both operations and the exact product-law rank $c{+}2$ are proved. The law is an ordinary typed

argument, not a metatheoretic assertion that arbitrary decoders commute definitionally with products.

Universe codes similarly have $u_a : C_0$ and a law

$$\mathsf{Eq}_1(\mathcal{U}_0, \mathsf{El}_0(u_a), C_a) : \mathcal{U}_2.$$

The two casts connect the decoded universe with its code carrier. `TarskiUniverses` proves formation and substitution naturality, as well as actual identity witnesses for **every external rank**, not merely finitely many computed examples.

**Theorem 9.1** (Identity realization of the local Tarski operations)**.** *Specialize*

$$C_a = \mathcal{U}_a, \qquad \mathsf{El}_a = \lambda X : \mathcal{U}_a.X, \qquad u_a = \mathcal{U}_a,$$

*and*

$$p_{a,b} = \lambda A : \mathcal{U}_a.\lambda B : (A \to \mathcal{U}_b).\Pi x : A.Bx.$$

*These are actual originally typed terms. Universe laws are inhabited by reflexivity using proved beta conversion. For each originally typed $A : \mathcal{U}_a$, $B : A \to \mathcal{U}_b$, $f : \Pi x : A.Bx$ and $u : A$, both sides of the product law reduce to $\Pi x : A.Bx$; reflexivity at that common product has the required law type by original conversion. The specialized casted abstraction and application are originally typed at $\Pi x : A.Bx$ and $B\,u$, respectively, and reduce to the literal $f$ and $f\,u$.*

`TarskiIdentity` proves the theorem by direct typing and actual beta reductions. It neither assumes the product law nor projects a precomputed normal semantic value. The closed identity product constructor and the closed universe-law witnesses are normal; specializing abstract decoded types may introduce administrative redexes, which are handled by proved conversion rather than subject expansion.

### 9.3 A finite dependent signature and its actual identity instantiation

The local declarations can now be assembled into an actual finite original context. Fix $k$. Use carriers through rank $k+1$, but product operations only for $a, b \le k$. Then $c = \max(a+1, b) \le k+1$ for every included pair. Requiring closure under products at **every** available carrier rank would instead keep demanding a further rank; the finite interface makes no such requirement. A sufficient literal ceiling is also still needed for a source containing $\mathcal{U}_r$: its result type $\mathcal{U}_0$ alone does not bound $r$.

The context is built in six dependent layers. Within each layer, every declaration refers only to the earlier layers; explicit weakening places it at its actual original de Bruijn scope.

| Fields | External indices | Number | Declaration sort |
|---|---|---|---|
| $C_r$ | $0 \le r \le k+1$ | $k+2$ | $0$ |
| $\mathsf{El}_r$ | $0 \le r \le k+1$ | $k+2$ | $1$ |
| $u_r$ | $0 \le r \le k+1$ | $k+2$ | $0$ |
| Universe-law witnesses | $0 \le r \le k+1$ | $k+2$ | $2$ |
| $p_{a,b}$ | $0 \le a, b \le k$ | $(k+1)^2$ | $a+2$ |
| Product-law families | $0 \le a, b \le k$ | $(k+1)^2$ | $c+2$ |

The last field is a single polymorphic witness for each rank pair:

$$\ell_{a,b} : \Pi A : C_a.\Pi B : (\mathsf{El}_a A \to C_b).\mathsf{Eq}_{c+1}\big(\mathcal{U}_c, \Pi x : \mathsf{El}_a A.\mathsf{El}_b(Bx), \mathsf{El}_c(p_{a,b}\, A\, B)\big).$$

Its declaration type has sort $c+2$: this absorbs the two extra binders because $c \geq a+1$. Thus infinitely many represented dependent families at a fixed rank pair need only one field, not an infinite context. `TarskiLawFamily` proves this sort calculation, literal substitution naturality and direct typing of the two applications $\ell_{a,b}\, A\, B$. Its actual closed normal identity inhabitant is

$$\lambda A : \mathcal{U}_a.\lambda B : (A \to \mathcal{U}_b).\mathsf{refl}_{c+1}(\mathcal{U}_c, \Pi x : A.Bx).$$

Original conversion assigns it the identity-specialized field type; its two applications reduce to the literal local reflexivity witness.

**Theorem 9.2** (Finite Tarski signature with constructive identity instantiation)**.** *For every $k$, `TarskiPalette` and `TarskiSignature` compute an original well-formed context $\Gamma_k$ of exactly $4(k+2)+2(k+1)^2$ declarations. Every declaration is beta-normal, and all six kinds of field have their stated original lookup typings within the indicated bounds. There is a computed simultaneous substitution $\sigma_k$ by actual closed identity terms, with a proved original typed-substitution judgment and exact equations for every field. No algebra-existence or law-inhabitation premise is supplied.*

`TarskiOperations` obtains product evidence by applying the law field; an individual equality witness is no longer an extra premise of each operation. Its product codes, casted abstraction and casted application have original typings in $\Gamma_k$. After actual replacement by $\sigma_k$, their computation laws include

$$\begin{aligned}
\sigma_k(p_{a,b}\, A\, B) &\longrightarrow^*_\beta \Pi x : \sigma_k A.(\sigma_k B)x,\\
\sigma_k(\mathsf{Abs}(A, B, f)) &\longrightarrow^*_\beta \sigma_k f,\\
\sigma_k(\mathsf{App}(A, B, g, u)) &\longrightarrow^*_\beta (\sigma_k g)(\sigma_k u).
\end{aligned}$$

The target is the literal substituted input, not an eta expansion or a precomputed source normal form. These are local operation equations; $f$ and $g$ have the respective decoded and encoded product types. The original `TarskiOperations` normality theorem additionally requires a normal decoded product annotation: a normal lambda-valued $B$ can create a redex in $B\, x$. The smart binder construction below removes this extra premise for its local operations. It does not by itself prove global quotation normality. The existing telescope machinery also closes any originally typed body over $\Gamma_k$, preserves normality of a normal body, and types actual application to all identity arguments before proving substitution recovery. This generic closure theorem does not construct a body representing an arbitrary source derivation.

### 9.4 Sort quotations and scope-correct dependent extensions

Write $w_r$ for the universe-law field from $\mathsf{El}_0(u_r)$ to $C_r$. `TarskiSorts` constructs the two views

$$\begin{aligned}
\mathsf{codeToTerm}_r(A) &= \mathsf{cast}^-_{0,\mathsf{El}_0(u_r)}(w_r)\, A : \mathsf{El}_0(u_r) \quad (A : C_r),\\
\mathsf{termToCode}_r(t) &= \mathsf{cast}^+_{0,\mathsf{El}_0(u_r)}(w_r)\, t : C_r \qquad (t : \mathsf{El}_0(u_r)).
\end{aligned}$$

For $r \leq k+1$ these are originally typed in $\Gamma_k$. They preserve normality of their inputs and, after actual identity substitution, reduce to the literal substituted input. These are specialization laws, not cancellation laws for abstract neutral casts.

In particular the source axiom $\mathcal{U}_r : \mathcal{U}_0$ has the normal quotation

$$\mathsf{sortQuote}_{k,r} = \mathsf{codeToTerm}_0(u_r) : \mathsf{El}_0(u_0).$$

The cast uses $w_0$, not $w_r$, because $u_r : C_0$. Closing this body over the finite signature gives an actual closed normal code. Its application to the identity arguments is originally typed at $\mathcal{U}_0$ and reduces to the literal $\mathcal{U}_r$. `closed_sort_run_typed` proves application typing independently of recovery, rather than invoking subject expansion. Every literal is covered by some finite $k$; this theorem concerns the sort fragment, not arbitrary source derivations.

`TarskiContext` extends the same signature by any finite sequence of independently sorted original declarations. It constructs the embedding of the signature variables, the specialized target context, and the actual typed substitution along this sequence. At each new binder the substitution is lifted:

$$\sigma^+(0) = 0, \qquad \sigma^+(i+1) = \uparrow(\sigma(i)).$$

Thus fresh variables remain variables; only the inherited signature fields receive their identity implementations. Both contexts are well-formed, original typing transports by this substitution, and each inherited field retains its lookup and identity equation. The signature embedding is injective. Normality of the extended abstract context requires normality of the added declarations; normality of its identity-specialized context is not asserted. Product and cast operations in this module accept arbitrary terms in the enlarged scope, including terms mentioning the new dependent variables, and retain original typing and identity recovery. These are concrete context-extension operations, not yet a translation of every original source context.

### 9.5 A normal local dependent binder

The troublesome family application in a decoded product is always to the freshly bound variable. `TarskiBinders.bound_apply` exploits precisely that case. In de Bruijn notation define

$$\mathsf{fresh}(B) = \begin{cases} b, & B = \lambda D.b, \\ (\uparrow B)\,0, & \text{otherwise.} \end{cases}$$

Then $(\uparrow B)\,0 \longrightarrow_\beta^* \mathsf{fresh}(B)$, and normal $B$ implies normal $\mathsf{fresh}(B)$. The lambda case merely removes the administrative fresh-variable application; it performs no arbitrary argument substitution. Renaming commutes exactly with this operation. Replacement has the correct one-way computation law

$$\sigma^+(\mathsf{fresh}(B)) \longrightarrow_\beta^* \mathsf{fresh}(\sigma(B)),$$

not a claimed literal equality: a neutral family may become a lambda.

Replace the decoded product annotation by

$$X^{\mathrm{smart}} = \Pi x : \mathsf{El}_a A.(\uparrow \mathsf{El}_b)\, \mathsf{fresh}(B).$$

The original decoded product reduces to this one. Original conversion therefore gives the existing law field the required equality type; no new law witness is assumed. Under the original input typing premises and $a, b \leq k$, the resulting smart abstraction and application are originally typed. They are normal whenever their respective inputs $A, B, f$ or $A, B, g, u$ are normal. In particular there is no separate normal-decoded-annotation premise. Their identity specializations recover the literal substituted $f$ and $g\,u$.

There is also a genuine local binder introduction. For $a, b \leq k$ and $A : C_a$ in $\Gamma_k$, suppose in $\Gamma_k, x : \mathsf{El}_a A$ that

$$B_0 : C_b, \qquad t : \mathsf{El}_b B_0.$$

Here the displayed signature fields are weakened into the extended scope. Set $B = \lambda x : \mathsf{El}_a A.B_0$ and $f = \lambda x : \mathsf{El}_a A.t$. The smart abstraction of $f$ has type $\mathsf{El}_c(p_{a,b}\, A\, B)$ and is normal when $A, B_0, t$ are normal. Its checked recovery target is the literal substituted lambda

$$\lambda x : \sigma_k(\mathsf{El}_a A).\sigma_k^+(t),$$

including that substituted decoded annotation, not an already contracted annotation $\sigma_k(A)$. For fixed $k, a, b$, equality of two such binder terms implies equality of all three supplied inputs $A, B_0, t$. On normal inputs this also holds for beta conversion. The supplied family remains present in the neutral law application even though the smart annotation contracts its administrative application. `binder_inputs_injective` and `binder_conversion_injective` establish this **local** fidelity, not injectivity of a full source-derivation translation. `ScopedTarski` now lifts these smart operations to any independently typed variable embedding of the finite signature. In particular, a concrete dependent context spine can be extended by $x : \mathsf{El}_a A$; the child environment and its lifted identity substitution are computed, not assumed. `nested_binder_typed` applies the binder rule at arbitrary depth using this actual child environment. Normality, literal identity recovery, exact renaming and weakening, and fixed-index local input/conversion injectivity are retained. Sort quotations and the code/term views also work in these scopes. This resolves the fixed-scope restriction of the earlier local interface. It does not yet translate an arbitrary source derivation: the represented domain, family and body remain independently supplied inputs.

### 9.6 Composing internal equality evidence

`IndexedCoherence` constructs two further original terms. For $e : \mathsf{Eq}_b(A, u, v)$ and $f : \mathsf{Eq}_b(A, v, w)$, composition is

$$\lambda P : (A \to \mathcal{U}_b).\lambda z : P\,u.f\,P\,(e\,P\,z) : \mathsf{Eq}_b(A, u, w).$$

For an ordinary nondependent map $g : A \to B$, congruence is

$$\lambda P : (B \to \mathcal{U}_b).e\,(\lambda x : A.P(gx)) : \mathsf{Eq}_b(B, gu, gv).$$

Both preserve the predicate level $b$, and commute literally with substitution. Composing two reflexivity witnesses or mapping reflexivity reduces to literal reflexivity, including the

latter's annotation. For arbitrary evidence, the identity laws are proved **after application** to a motive and an argument; no eta contraction of $\lambda P.\lambda z.e\,P\,z$ to $e$ is claimed. These witnesses supply composition and nondependent congruence, not an extensionality principle for dependent families.

### 9.7 Predicate levels for reversing and joining evidence

An internal forward join must still reverse its second half. The equality encoding does not silently supply symmetry at the same predicate level. For $A : \mathcal{U}_a$ and desired output level $d$, set

$$h(a, d) = \max(a + 2, d + 1).$$

This is the formation rank of $\mathsf{Eq}_d(A, u, v)$. `IndexedEqualityLevels` constructs the actual original terms

$$\begin{aligned}\mathsf{lower}_d(e) &= e\,(\lambda x : A.\mathsf{Eq}_d(A, u, x))\,\mathsf{refl}_d(A, u),\\ \mathsf{reverse}_d(e) &= e\,(\lambda x : A.\mathsf{Eq}_d(A, x, u))\,\mathsf{refl}_d(A, u).\end{aligned}$$

Given $e : \mathsf{Eq}_{h(a,d)}(A, u, v)$, their respective types are $\mathsf{Eq}_d(A, u, v)$ and $\mathsf{Eq}_d(A, v, u)$. Both reduce to literal level-$d$ reflexivity when $e$ is reflexivity. For $e : u \sim w$ and $f : v \sim w$ at level $h(a, d)$, composing $\mathsf{lower}_d(e)$ with $\mathsf{reverse}_d(f)$ gives $\mathsf{Eq}_d(A, u, v)$. These constructors commute literally with replacement and are normal on normal inputs with non-lambda-headed evidence. They are not asserted normal after arbitrary specialization.

For equality between types $X, Y : \mathcal{U}_c$, the carrier is $\mathcal{U}_c : \mathcal{U}_0$. Consequently, predicate level $c + 2$ suffices to reverse into the level $c + 1$ used by the casts above. This is explicit rank accounting, not cumulativity or a claim that any larger predicate level is interchangeable. Since forward composition preserves the predicate level, one can compose each forward path before reversing once; this operation does not itself require a rank increase at every path step. Generating the input witnesses and proving that a chosen finite palette supplies their exact levels remain separate obligations.

### 9.8 Synchronized evidence for dependent application

For $A : \mathcal{U}_a$ and $B : A \to \mathcal{U}_b$, define graph equality by

$$\mathsf{GEq}_d(A, B, u, y, v, z) = \Pi P : (\Pi x : A.Bx \to \mathcal{U}_d).P\,u\,y \to P\,v\,z.$$

Here $y : Bu$ and $z : Bv$ may live in different fibers. Its exact sort is $\mathcal{U}_{\max(\max(a+2,b+2),d+1)}$. `DependentEquality` constructs reflexivity, direct graph transport, and synchronized dependent congruence: for $f : \Pi x : A.Bx$ and $e : \mathsf{Eq}_d(A, u, v)$,

$$\lambda P.\,e\,(\lambda x : A.P\,x\,(f\,x)) : \mathsf{GEq}_d(A, B, u, f\,u, v, f\,v).$$

The predicate annotation on $P$ is the one displayed in the definition. This preserves level $d$ and needs neither function extensionality nor a proof-dependent eliminator. At reflexivity it reduces to literal graph reflexivity, including its annotation. For a fixed index, ordinary evidence $e : \mathsf{Eq}_d(Bu, y, z)$ also gives

$$\lambda P.\, e\,(P\,u) : \mathsf{GEq}_d(A, B, u, y, u, z).$$

Both constructions commute with replacement. Their normality theorems retain the necessary non-lambda-head guards: a normal lambda-valued family $B$ still introduces a redex in the predicate annotation $B\,x$. This graph evidence does not establish equality in the target fiber between $f\,v$ and the result of a separately chosen transport of $f\,u$. Nor does it establish function equality from arbitrary pointwise evidence. It tracks the dependent index and value together.

### 9.9 Binder congruence through a common map

There is a useful binder case that needs no pointwise-to-function principle. Let $D : \mathcal{U}_s$, $X : \mathcal{U}_a$, $C : \mathcal{U}_c$, and suppose $b(z, x) : C$ in the independently typed context $z : D, x : X$. The type $X$ and codomain $C$ do not depend on $z$, but the body may depend on both variables. Define

$$H = \lambda z : D.\lambda x : X.b(z, x).$$

For an **outer** witness $e : \mathsf{Eq}_d(D, u, v)$, ordinary map congruence constructs

$$\mathsf{Eq}_d\big(X \to C, \lambda x : X.b(u, x), \lambda x : X.b(v, x)\big).$$

`BinderCongruence` proves this at sort $\mathcal{U}_{\max(\max(a+1,c)+2,d+1)}$, with actual binder-aware substitution, replacement naturality and literal reflexivity computation. Taking $X = \mathsf{El}_a A$ and $C = C_b$, then mapping by $p_{a,b}\,A$, yields internal equality of the corresponding product codes. At the actual identity product constructor and reflexive input, its witness reduces to literal reflexivity at $\Pi x : A.b(u, x)$, without eta conversion.

This handles families obtained from a common map and one witness outside the binder. It does not handle arbitrary $x$-dependent evidence $e_x$. The generated proof terms can contain administrative beta redexes; full normality is not claimed for this constructor.

### 9.10 An independently realized beta-law interface

The neutral-cast counterexample does not prevent adding explicit internal evidence to a finite algebra, provided its identity interpretation is constructed independently. `TarskiBeta` makes this option concrete. Put $c = \max(a + 1, b)$ and $e = \ell_{a,b}\,A\,B$. For $f : \Pi x : \mathsf{El}_a A.\mathsf{El}_b(Bx)$ and $u : \mathsf{El}_a A$, the proposed field concludes

$$\mathsf{Eq}_d\big(\mathsf{El}_b(Bu), \mathsf{App}(\mathsf{Abs}(f), u), f\,u\big).$$

Here $\mathsf{Abs}$ and $\mathsf{App}$ are the original two casts using $e$, not silently cancelling operations. Binding $A, B, f, u$ gives an original PTS type of sort

$$\mathcal{U}_{\max(c+1,\max(b+2,d+1))}.$$

The declaration is normal when its carrier, decoder and law fields are variables. Under the actual identity interpretation it has the closed normal inhabitant

$$\lambda A : \mathcal{U}_a.\lambda B : (A \to \mathcal{U}_b).\lambda f : (\Pi x : A.Bx).\lambda u : A.\mathsf{refl}_d(Bu, f\,u).$$

Its typing is proved at the literal specialized field type by original conversion. Four actual applications reduce to literal reflexivity, including its annotation. Formation of this field is not a proof that an arbitrary product-law witness already entails it.

`TarskiBetaSignature` adds one such field for each $a, b, d \leq k$ to the unchanged six-layer signature. The resulting actual context has

$$4(k+2) + 2(k+1)^2 + (k+1)^3$$

declarations. Its original well-formedness, normal declarations, variable lookups and constructive typed identity substitution are proved. Applying a field to independently typed $A, B, f, u$ yields the displayed internal equality; substituting the actual identity fields reduces this application to reflexivity on the literal substituted inputs. The inherited signature embedding supplies the scoped smart operations' typing and normality interface. These finite beta laws neither add a primitive to PTS-rk nor prove the full source-conversion theorem. In particular, they do not yet supply congruence under dependent binders or a family-level equality from pointwise evidence.

`TarskiGraphBeta` connects this actual finite field to graph equality. It constructs the decoded fiber $F = \lambda x : \mathsf{El}_a A.\mathsf{El}_b(Bx)$ and uses the fixed-fiber bridge to obtain

$$\mathsf{GEq}_d\big(\mathsf{El}_a A, F, u, \mathsf{App}(\mathsf{Abs}(f), u), u, f\,u\big).$$

Typing uses the real field lookup and original conversion for $F\,u \longrightarrow_\beta \mathsf{El}_b(Bu)$; no new equality premise replaces the field. Actual identity substitution reduces the witness to graph reflexivity at the literal substituted domain, family, index and value. In particular the substituted lambda family is retained, not eta-contracted to $B$. Its filled witness is originally typed directly from the constructed typed substitution. This literal graph specialization is not asserted normal: its lambda-valued fiber creates the annotation redex just noted.

## 9.11 Executable full-source derivations and canonical conversion traces

`FullSource` gives computational evidence in Rocq's `Type` for the exact original six typing rules, single beta steps, forward reductions and bidirectional conversion paths. The terms, contexts and lambda annotations are the original syntax. Inhabitation of each datatype is proved equivalent to the corresponding original relation; the typing equivalence covers all original judgments, without a well-formed-context restriction. Conversion paths may have untyped intermediate terms, just as the original relation permits. A computable finite ceiling bounds the supplied derivation's literal labels, its explicitly recorded rule ranks and every conversion intermediate's literals. This is not yet a sufficient target-quoter budget or a bound on every inferred semantic rank.

The distinction between supplied data and erased proofs matters. The equivalence is propositional: no function extracts a computational derivation from an arbitrary original `Prop` proof. Subsequent executable operations consume supplied derivation data.

`FullConversion` computes forward traces for every greedy pass and for the existing automatic open-context normalizer. It compares the two computed endpoints by structural equality of annotated syntax. Success returns actual traces

$$t \longrightarrow_\beta^* v \quad \text{and} \quad u \longrightarrow_\beta^* v.$$

Success always implies original conversion, even on raw inputs. In a well-formed original context, for independently originally typed $t : T$ and $u : U$, the comparison succeeds exactly when $t =_\beta u$. Its common endpoint is normal, and every stage of each forward trace retains its starting term's original type, $T$ or $U$ respectively. No claim of completeness on arbitrary untyped syntax is made.

`CanonicalSource.canonicalize` recursively transforms a supplied full source derivation in a well-formed context. At each conversion rule it replaces the supplied zigzag by this computed forward join, leaving **all source terms, types, contexts and annotations literally unchanged**. The function obtains its data from the executable comparison; validity and the supplied conversion proof only exclude the failure branch. An independent output theorem certifies normal common endpoints and originally typed intermediate stages at every conversion node. Every original typing derivation in a well-formed context has such certified data. Thus a future translation can traverse typed forward paths without interpreting arbitrary untyped conversion intermediates. These are metalevel source certificates, not object-language conversion witnesses or a target quotation. Literal source preservation leaves the existing syntax-injective HOAS coding available, but does not prove injectivity of a dependent PHOAS translation by itself.

`SourceSubstitution` now supplies an executable source-data operation needed for this induction. A pointwise substitution carries independent `FullSource` derivations for its variable images; inhabitation of such data is equivalent to the original typed-substitution relation, again only propositionally. Structural renaming, weakening, binder lifting and substitution transform derivations and their explicit conversion paths. Given body and argument derivation data, `beta_contractum_derivation` computes a derivation of the literal $b[u/x] : B[u/x]$, including substituted annotations. Supplying the domain formation also constructs the typed redex and proves its original beta step and contractum typing. This does not infer derivation data from the propositional stage typings above, or by itself construct the target quotation's substitution witness. Typed constructor-view substitution is supplied later.

## 9.12 Why the local signature alone is insufficient

The finite signature and its identity substitution do not alone construct a quotation of every original typing derivation with generated substitution and source-conversion coherence. In particular, arbitrary proof-headed casts do not definitionally cancel. A pointwise Leibniz witness for two families also cannot simply be used as an equality of the object-language functions expected by congruence of $p_{a,b}$.

`TarskiRoundtrip` makes the first limitation precise for the existing cast definitions. In the actual well-formed original context

$$S : \mathcal{U}_c, \quad T : \mathcal{U}_c, \quad e : \mathsf{Eq}_{c+1}(\mathcal{U}_c, S, T), \quad x : S,$$

both $x$ and $\mathsf{cast}^-_{c,S}(e)(\mathsf{cast}^+_{c,S}(e)\, x)$ are originally typed at $S$. They are distinct normal terms, hence are not beta-convertible. Extending this context by $K : S \to \mathcal{U}_0$ gives two normal, well-typed, nonconvertible type indices by applying $K$ to them. At actual reflexivity evidence the same roundtrip instead reduces to $x$. This counterexample forbids treating these neutral casts as a definitional retraction. It does not exclude internal equality evidence, additional independently realized coherence fields, different casts, or full dependent PHOAS.

There is a checked beta-only regression. With $D = \mathcal{U}_0 \to \mathcal{U}_0$, the closed normal functions

$$F = \lambda f : D.f, \qquad G = \lambda f : D.\lambda x : \mathcal{U}_0.f\,x$$

have the same original type but are not beta-convertible. Nevertheless, $F\,f\,x$ and $G\,f\,x$ both reduce to $f\,x$. In an original well-formed context containing $f : D$ and $K : D \to \mathcal{U}_0$, the types $K\,f$ and $K\,(\lambda x : \mathcal{U}_0.f\,x)$ are likewise normal, well-typed and nonconvertible. `EtaBoundary` proves these facts from normal-form uniqueness. They forbid silently replacing beta conversion by pointwise agreement; they do not rule out full dependent PHOAS or prove that every possible internal equality witness is absent.

A construction based only on these local laws would need to translate the executable source data using the arbitrary-scope sort and smart binder operations, and generate the needed internal substitution and conversion witnesses along the computed typed forward paths. It must establish their identity computation laws, global annotated normality and source-syntax fidelity. The finite beta fields and their graph specialization address a local application/abstraction law. Common-map binder congruence and synchronized dependent application evidence do not give dependent congruence for every source reduction. Metalevel canonicalization, raw syntax fidelity and the local casts remain separate proved ingredients, not substitutes for that induction. The checked equality-layer construction below completes full-source coverage by a different repair interface.

### 9.13 Contextual beta laws and a finite telescope interface

The specific cast-beta equation under a dependent telescope is now constructed. This is an independently realized law family, not a rule turning arbitrary pointwise equality into function equality. Let

$$\Delta = (x_1 : X_1, \ldots, x_m : X_m), \qquad \Gamma, x_1 : X_1, \ldots, x_{i-1} : X_{i-1} \vdash X_i : \mathcal{U}_{r_i}.$$

`ContextualTelescope` records these original formation judgments separately from any quotation property. Its closures are literal annotated products and lambdas. For a body type of rank $b$, product closure has exact rank

$$R(\vec{r}, b) = \max\big(b, \max(\{0\} \cup \{r_i + 1 : 1 \le i \le m\})\big).$$

The module proves original closure typing, renaming, substitution and annotation-sensitive normality. In particular, closing a normal body is normal precisely when every retained domain annotation is normal.

For the previously defined cast operations, `ContextualBeta` forms

$$\mathsf{Eq}_d\big(\Pi\Delta.\mathsf{El}_b(B\,u), \lambda\Delta.\mathsf{castApp}(e, \mathsf{castAbs}(e, f), u), \lambda\Delta.f\,u\big).$$

Its sort rank is $\max(R(\vec{r}, b) + 2, d + 1)$. The typing proof uses the supplied original typings of the carriers, decoders, product code, law, function and argument. At the actual identity interpretation, the two endpoints reduce to the same literal multi-lambda. Reflexivity at that term is constructed and typed independently, then assigned the abstract equation type by original type conversion. No subject expansion of the witness, eta rule or function-extensionality assumption is used.

`TelescopeBetaFamily` additionally binds four object arguments: the carrier family $A$, dependent code family $B$, function family $f$, and argument family $u$, each abstracted over $\Delta$. Writing $A_\Delta$ for application to all variables of $\Delta$, the conclusion uses $e = \ell\, A_\Delta\, B_\Delta$, $f_\Delta$ and $u_\Delta$. The full four-argument family has formation rank

$$\max\big(\max(\{0\} \cup \{r_i + 1\}) + 2, a + 2, b + 2, d + 1\big).$$

Its identity implementation uses plain, reduced lambda annotations and ends in reflexivity at $\lambda\Delta.f_\Delta\, u_\Delta$. Direct original typing, full annotated normality under normal domains, substitution and literal four-argument specialization are proved separately. The one-binder `ContextualBetaFamily` also quantifies its domain $X : \mathcal{U}_s$ and has exact rank $\max(s + 3, a + 2, b + 2, d + 1)$.

Keeping a concrete $\Delta$ in an external field index would not give a finite interface. `UniformTelescope` instead generates bound domain parameters

$$X_1 : \mathcal{U}_{r_1}, \quad X_2 : \Pi x_1 : X_1.\mathcal{U}_{r_2}, \quad \ldots, \quad X_m : \Pi x_1 : X_1.\cdots\Pi x_{m-1} : X_{m-1}\,\vec{x}_{<m-1}.\mathcal{U}_{r_m}.$$

The generated telescope uses neutral applications $X_i\,\vec{x}_{<i}$ as its domain annotations. Its parameter context and its domains have original formation and full normality, proved for every rank vector. Parameter $X_i$ has declaration-formation rank $R((r_1, \ldots, r_{i-1}), 0)$, since $\mathcal{U}_{r_i} : \mathcal{U}_0$. Renaming and substitution commute with the generator exactly.

`UniformTelescopeRecovery.realize` computes actual parameter values from any supplied concrete telescope: each declaration is lambda-closed over its literal preceding domains. Independent original formation suffices to prove the resulting substitution is typed. Instantiated generated domains beta-reduce annotation by annotation to the concrete telescope, expressed in the generator's accumulated scope order. A further theorem recovers the exact original product closure ending in any universe $\mathcal{U}_s$. This is genuine domain-parameter coverage. The subsequent `OpenTelescopeRecovery` module also transports arbitrary open bodies and literal lambda closures, with original typing in both directions across the realized telescope. `ConcreteContextualBeta` applies the actual finite field to independently typed open arguments and proves literal identity recovery, normality under explicit normality guards, and local input injectivity. Integration with recursive quotation is a further obligation.

`UniformContextualBeta` abstracts the four-argument family over these domain parameters. Its syntax depends externally only on $\vec{r}, a, b, d$, not on the concrete domain terms. With all these ranks bounded by $k$, its declaration has formation rank at most $k + 3$. Its actual identity value is a closed normal original PTS term; substitution invariance, original typing and normality are separate proofs.

Length must be bounded independently. `ContextualBudgets` proves that vectors of arbitrarily large length can have closure rank 1: take arbitrarily many zero-ranked declarations. Consequently no bound on this rank alone bounds telescope length. It also gives an executable complete enumeration of the vectors of length at most $L$ with entries at most $k$, of cardinality $\sum_{j=0}^{L}(k+1)^j$. `TelescopeBetaSignature` adds exactly

$$(k + 1)^3 \sum_{j=0}^{L}(k + 1)^j$$

contextual fields to the existing finite beta signature. Executable lookup recovers each bounded vector and rank triple. Context formation, full annotation normality, field typing, the actual typed identity substitution, and closed packaging/application recovery are proved. Every finite vector and triple has a computed sufficient budget; this descriptor coverage is not yet coverage of translated full source derivations. The separate one-binder signature retains local input-syntax and conversion injectivity. Existing faithful raw-HOAS and STLC quotations are unchanged.

These results discharge the formation, independent identity-realization, and finite-budget obligations for this contextual cast-beta family. They do not complete dependent source-reduction coherence: assembling compatible context/type/term/substitution translations is a separate obligation, supplied by the later checked-layer construction.

### 9.14 Checked equality layers and finite constructor views

A second construction strategy uses an acyclic sequence of independently realized equality-only signature extensions. Suppose a previously formed signature $\mathcal{S}$ has a closed, originally typed identity filling $\iota$, and $\Delta$ is an originally formed local dependent telescope. An extension may contain the field

$$e : \Pi\Delta.\, \mathrm{Eq}_d(C, u, v)$$

only after $C$, $u$ and $v$ are independently typed in the old context, all field annotations are normal, and the actual filled endpoints $u[\iota]$ and $v[\iota]$ are beta-convertible. These are decidable checks on ordinary annotated PTS syntax, not tests for quotation success. `OriginalTypecheck` supplies executable inference and checking with soundness and completeness for the unchanged original typing judgment in well-formed contexts. `BoundedSyntaxEnumeration` enumerates every syntax tree within explicit node and literal ceilings, including annotations.

`RealizedEquality` constructs the equality realizer using the common normal form of the filled endpoints. `ContextualEqualityRealizer` closes it over the local telescope and normalizes this eraser program. This does not normalize the represented payload. Its lambda-spine invariant proves that application to all telescope arguments, a motive and a payload reduces to that literal payload. `ContextualEqualityExtension` and `CodeAlignment` therefore provide actual originally typed code alignment, including under dependent binders, with normal abstract applications and literal filled recovery. No arbitrary semantic-value fields are added.

`CheckedEqualityFields` constructs a complete finite checked layer over a fixed preceding context. `CheckedContextualFields` extends its candidate syntax to locally dependent telescopes and proves actual field lookup, original formation, normality, closed typed filling and literal transport erasure. Its coverage theorem bounds telescope length, node counts and literal labels separately. The whole interpreter below is indexed by a finite vector of these numerical layer budgets; each layer refers only to earlier layers. This is a stronger numerical index than rank alone. Neither finite candidate coverage nor the existing rank bounds already establish coverage of all translated source derivations.

The source recursion must also terminate without re-quoting an ever larger generated type derivation. `SourceValidity` computes original result-type formation, but naively recursing on that output is unsuitable: substitution can enlarge it, and $\mathcal{U}_0 : \mathcal{U}_0$ creates a terminal-sort cycle. `QuotationTree` instead keeps a finite literal source-constructor tree separate from its target root cache. A Pi view retains independently typed domain and codomain children even when conversion has changed that cache. `QuotationCache`, `QuotationViewRenaming` and

`QuotationViewSubstitution` make this separation typed. Their executable structural substitution keeps the root cache literally substituted. If a source variable is replaced by a Pi term, it takes the argument's stored constructor shape and checks the new root using original typing and uniqueness of sort ranks; it does not call quotation on a synthesized source derivation. Under a fresh binder, the actual source and target realizations are extended constructively. The direct codomain operation produces a view of literal $B[u/x]$ with root cache exactly $B^q[q/x]$. Normal neutral substitution preserves full normality, including all annotations.

## 9.15 A dependent decoder with an ordinary represented-type argument

`DependentProgramDecoder` separates the decoder interface from the structural all-source quotation construction below. Let $\mathcal{S}$ be a finite originally formed closed signature, with declaration-rank vector $\vec{r}$, a code carrier $C : \mathcal{U}_0$, a decoder $E : C \to \mathcal{U}_s$, and an independently typed closed filling $\iota$. Define

$$\begin{aligned} \mathsf{TyProg} &= \Pi\mathcal{S}.C, \\ \mathsf{TmProg}(P) &= \Pi\mathcal{S}.E\,(P\,\mathcal{S}), \\ \mathsf{Result}(P) &= E[\iota]\,(P\,\iota), \\ \mathsf{Dec} &= \lambda P : \mathsf{TyProg}.\lambda Q : \mathsf{TmProg}(P).Q\,\iota. \end{aligned}$$

Here $P$ is an ordinary object-language argument, not an external source type or typing-derivation index. In the numerical family below, $\mathcal{S}, C, E, \iota$ are determined by its numerical signature index and result rank. `TelescopeInstantiation` proves the original typed mixed substitution that retains $P, Q$ while filling the signature parameters. The decoder is directly originally typed; no subject expansion is used. Its type has formation rank

$$\max\big(R(\vec{r},0)+1,\ R(\vec{r},s)+1,\ s\big).$$

If the signature annotations, $C$, the neutral $E$, and all filling values are normal, the decoder itself is fully normal. Given independently typed open codes $A^q : C$ and $t^q : E\,A^q$, their actual lambda closures give $P$ and $Q$. The typing of $Q : \mathsf{TmProg}(P)$ uses conversion of its type, whose application spine reduces to the original decoded code type; the payload is unchanged. Two administrative beta steps and signature application yield

$$\mathsf{Dec}\,(\lambda\mathcal{S}.A^q)\,(\lambda\mathcal{S}.t^q)\ \to^*\ t^q[\iota].$$

Thus any subsequently constructed literal recovery of $t^q$ is preserved. Likewise, when $E[\iota]$ is the type identity and $A^q[\iota] \to^* A$, the result type reduces to $A$. Closure preserves normality and is syntactically injective at a fixed signature. These generic decoder and packaging theorems do not alone provide full dependent quotation; the following structural construction does. Strong recovery alone would not prove syntax-level injectivity for distinct beta-convertible sources.

## 9.16 Faithful packages of actual dependent programs

`FaithfulProgramDecoder` separates evaluation from syntax fidelity. Let $\mathsf{Tag}_r$ be the existing closed raw HOAS code type, and write $\langle t\rangle_r$ for its normal, annotation-preserving quotation. For an actual represented program $Q : V$, define

$$\begin{aligned}\mathsf{Pack}_r(V) &= (V \to \mathsf{Tag}_r \to V) \to V,\\ \mathsf{pack}_r(V,Q,t) &= \lambda h : (V \to \mathsf{Tag}_r \to V).h\,Q\,\langle t\rangle_r,\\ \mathsf{pick}_r(V) &= \lambda q : V.\lambda z : \mathsf{Tag}_r.q.\end{aligned}$$

The faithful decoder first selects $Q$ and then applies the actual signature filling to it. Thus its computational result is $Q\,\iota$, not $Q$ itself and not an externally computed normal form of $t$. Only generic packaging lemmas are reused from `WeakFullInterpretation`; neither its source normalizer nor its precomputed semantic payload is used. The normal annotation $V = \Pi\mathcal{S}.E\,A^q$ is convertible to $\mathsf{TmProg}(P)$ for $P = \lambda\mathcal{S}.A^q$. Conversion changes the package's type, not its payload or annotations.

The faithful decoder's type has exactly the checked formation rank

$$\max\big(R(\vec{r},0)+1, \max(R(\vec{r},s)+2, r+5)+1,\ s\big).$$

Normal actual signature annotations and filling values make the decoder normal; normal $V, Q$ make the package normal. Equality of two such package terms implies literal equality of their annotated source terms, even when their signatures, derivations and source types differ. Conversion of normal packages has the same consequence by confluence. The tag establishes this fidelity independently of any possible noninjectivity of conversion repairs.

`ClosedQuotationPackage` combines these results with an independently constructed dependent quotation judgment. It closes the actual signature, proves original typings for the type program, faithful term package and interpreter, and proves reduction to the literal source term and source type. `NormalEqualityFill` derives the required filling normality for the actual finite Tarski signature and every checked equality layer; it is not an extra condition imposed on source terms. These are closing theorems; the structural construction below supplies the required quotation for every original derivation.

## 9.17 Executable rank budgets and literal dependent local scopes

`SourceQuotationBudget` computes a budget from Type-valued original derivation and context data. It takes maxima over all recursive premises, their result-type ranks, explicit binder ranks and literal sort labels. The resulting bound covers the extended contexts in both binder cases and both premises of application and conversion. Literal labels cannot be replaced by result-type ranks: $\mathcal{U}_{37} : \mathcal{U}_0$ has result-type rank zero, but its literal ceiling is 37. This regression is checked explicitly. Original proposition-valued typing has a coverage theorem; no typing proof is eliminated from `Prop` to compute quotation data.

`LocalQuotationSpine` keeps source binders in snoc order. A state with $n$ local binders and $m$ global fields has target scope $n+m$ and literal source scope $n$. Pushing a binder therefore has scope $n+1$ without normalizing its source declaration. Global equality layers can grow while preserving that same literal source context. `LocalSpineNaturality` proves that the telescope allocator's presentation commutes with such growth; its proof-only scope casts introduce no object-language constructors.

The execution plan has only three constructors: the empty plan, a request carrying three numeric bounds, and sequential composition. Its evaluation uses the actual intermediate realized signature. Flattening yields the same finite vector of numeric layer budgets. This is a source-independent index format, not a rank-only uniformity theorem, and no elementary

quotation-cost bound is asserted. The next construction discharges the full structural source-quotation obligation.

### 9.18 Full dependent interpretation with numerical signature plans

The structural construction is `RecursiveQuotation.quote_derivation`. Its inputs are an independent `FullSource.derivation`, original context data, an actually realized local state and variable environment, and the computed numerical budget. Its result is a finite plan and a quotation judgment in the corresponding extended signature. The result judgment contains independently checked original typing, full normality, a finite typed constructor view, and reduction of the actual filled program to the literal annotated source. None of those target properties is an input assumption about the source derivation.

The recursion has six cases, matching the original rules.

- Sorts and variables use the actual finite interface and variable environment.
- Products and lambdas quote the domain, extend by its normal decoded code, and recursively quote the existing body premise under the literal source binder. `FullQuotation` proves that global signature growth commutes with this local extension.
- Application inverts the stored literal product view, not the possibly converted root cache. `ApplicationQuotation` allocates exactly two checked layers: one aligns the argument's type code, and one aligns the function's root with its constructed product code. Structural substitution of the codomain view then gives the actual dependent result type.
- Conversion quotes the rule's existing target-formation premise and aligns the original payload with that independently constructed code. Its equality realizer erases to the same literal payload.

Quotation of a term whose type is a literal universe uses a separate nonrecursive terminal repair. It does not recursively quote a generated validity derivation; in particular, $\mathcal{U}_0 : \mathcal{U}_0$ creates no recursive cycle. No untyped intermediate of a source conversion zigzag needs quotation.

Write $I_{r,k,p,s}$ for the family in `IndexedProgramInterpreter`. Here $k$ is the finite source-interface budget, $p$ a pure numerical allocation plan, $s$ the result-code rank, and $r$ the raw-tag carrier rank. These determine the signature, its identity filling and the interpreter. The represented type $P$ and faithful term program $Q$ are ordinary inputs.

**Theorem 9.3** (Full closed original-source interpretation)**.** *For every original judgment $\vdash t : T$ and every tag rank $r$, there exist finite numerical indices $k, p, s$ and closed original terms $P, Q$ such that $s \le k+1$, $P, Q, I_{r,k,p,s}$ are fully beta-normal, and*

$$\begin{aligned}
&\vdash P : \mathsf{TyProg}_{k,p,s},\\
&\vdash Q : \mathsf{Pack}_r(\mathsf{TmProg}_{k,p,s}(P)),\\
&\vdash I_{r,k,p,s} : \Pi P : \mathsf{TyProg}_{k,p,s}.\mathsf{Pack}_r(\mathsf{TmProg}_{k,p,s}(P)) \to \mathsf{Result}_{k,p,s}(P),\\
&\vdash I_{r,k,p,s}\, P\, Q : T,\\
&I_{r,k,p,s}\, P\, Q \to^* t.
\end{aligned}$$

*The interpreter type has the explicit finite formation rank proved above. Applying the type program to the identity filling reduces to the literal source type $T$. For the constructed quotation procedure, equality or beta conversion of two term programs implies literal equality of their annotated source terms, independently of their derivations and numerical plans.*

*Proof.* For supplied Type-valued derivation data, compute `SourceQuotationBudget`, initialize the independently realized empty-source state, and run structural quotation. Close its resulting judgment using `ClosedQuotationPackage`. Actual signature and filling normality follow from the checked layer construction; the faithful envelope supplies syntax injectivity. For the propositional coverage theorem, original typing implies inhabitation of the independent derivation datatype. This witness is used only inside the final proposition, not eliminated from `Prop` to compute a program. The endpoints are `FullDependentInterpreter.every_closed_original`, `closed_type_recovers`, `closed_injective` and `closed_conversion_injective`. □

`SourceClosure` extends coverage to every originally well-formed open context $\Gamma$. It computes the literal closure $\lambda\Gamma.t$ and its original derivation, quotes this closed term, then applies the interpreted closure to the ordinary variables of $\Gamma$. `FullDependentInterpreter.every_original` proves original result typing and literal recovery. Syntax fidelity is retained by `program_injective`; conversion fidelity is retained by `program_conversion_injective`. Both preserve source syntax at a fixed source scope, even across different same-length contexts. This open interface uses explicit closure and reopening; it is not a claim that a fixed closed head can create arbitrary free variables.

This is a positive **numerical-plan-indexed full-source** result. Plan length and syntax bounds may grow with the source derivation. It does not establish one decoder for all terms of a fixed rank, a single all-rank interpreter, efficient uniform enumeration, or an interpreter for unadorned raw HOAS without the dependent PHOAS program. The interpreter evaluates that actual program; the raw tag is used only for fidelity. Extraction and direct Rocq evaluation are distinct operational claims: proof-only equality transports can obstruct direct reduction of dependent records, and no general runtime benchmark follows merely from the structural definition.

A proof-erased OCaml smoke test executes the actual closed sort-quotation path through this construction and checks an empty allocation plan and normal type and term programs. Its successful run checks that leaf path, not the performance of dependent application or large equality layers.

Every originally typed member of this interpreter family is itself a closed source term, so the coverage theorem applies to it at some finite numerical index. That index may be larger or different: this consequence is not a same-index fixed-point or a rank-only self-applicability claim.

# 10 Executable structural quotation for the simply typed fragment

The existing independent source `ChurchSTLC` has a distinguished ground type variable $o : \mathcal{U}_0$, simple types $\sigma$, and annotated lambda terms. It has its own typing rules. Its embedding uses

$$|o| = o, \qquad |\sigma \to \tau| = |\sigma| \to |\tau|, \qquad \mathrm{ord}(\sigma \to \tau) = \max(\mathrm{ord}(\sigma) + 1, \mathrm{ord}(\tau)).$$

All type codes are beta-normal, and $|\sigma|$ inhabits $\mathcal{U}_{\mathrm{ord}(\sigma)}$ in the target context. No source rule depends on conversion of terms inside types.

Fix a budget $d$ bounding the order of the **function type of every application** in a source typing derivation. It is not enough to bound the final result type. For an example entirely within STLC, put $\sigma_0 = o \to o$, $w_0 = \lambda z : o.z$, $\sigma_{N+1} = \sigma_N \to \sigma_0$, and $w_{N+1} = \lambda x : \sigma_N.w_0$. Then $(\lambda x : \sigma_N.w_0)\, w_N$ has the fixed result type $\sigma_0$, while its function and argument orders grow without bound. The independent relation `bounded_type` adds only this numerical bound to the original STLC rules; `bounded_type_source` forgets it, and `every_source_has_budget` proves that every source derivation has some finite budget. No quotation or recovery property defines the source domain.

Introduce variables $b_0 : \mathsf{I}_0, \ldots, b_d : \mathsf{I}_d$. The prequotation is the following derivation-directed structural translation:

$$\begin{aligned} q(x) &= x, \\ q(\lambda x : |\sigma|.t) &= \lambda x : |\sigma|.q(t), \\ q(f\, u) &= b_r\, |\sigma \to \tau|\, q(f)\, q(u), \qquad r = \mathrm{ord}(\sigma \to \tau), \end{aligned}$$

where the last case uses the source judgment $f : \sigma \to \tau$. The formal version supplies all scope changes and binder shifts explicitly. No application in $q(t)$ has a lambda at its head: each translated application is headed by a neutral blocker. The remaining annotations are normal simple-type codes. Thus the quotation is normal for the **full original PTS reduction**, including annotation reduction.

Typing follows from $b_r\, |\sigma \to \tau| : (|\sigma| \to |\tau|) \to (|\sigma| \to |\tau|)$ and the translated subderivations. Substituting $\mathsf{id}_r$ for each blocker gives strong recovery to the original annotated source term by induction. Syntactic injectivity is proved at each fixed budget with a structural eraser for the administrative blocker pattern. Recovery up to beta conversion alone would not establish injectivity, since distinct source terms may be beta-convertible.

The Rocq module `RankedQuotation` implements these claims for arbitrary bounded derivations and explicitly typed blocker contexts. Its original existence theorem is relational, in Rocq `Prop`. The theorem `finite_budget_quotation` supplies the actual finite blocker context, and `finite_package_injective` proves injectivity of the resulting packages. `every_source_has_normal_representation` covers every independent STLC derivation at some budget.

The new `ExecutableQuotation` module supplies an executable external algorithm. Its source derivation is an inductive datatype in Rocq `Type`, with the independent variable, lambda, and application rules, indexed by the original annotated source term and its simple type. No constructor assumes target PTS typing, quotation, or recovery. The theorem `source_derivation` relates its inhabitation to the earlier independent `ChurchSTLC.has_type` judgment. This is not an illicit elimination from a proof in `Prop` into computational data: the executable function consumes the derivation data explicitly. The coverage theorem remains a proposition; it is not a source type-inference algorithm.

Structural recursion computes both prequotation and a sufficient budget:

$$\begin{aligned} b(D_{\mathrm{var}}) &= 0, & b(D_{\mathrm{lam}}) &= b(D_{\mathrm{body}}), \\ b(D_{\mathrm{app}}) &= \max\{\mathrm{ord}(\sigma \to \tau), b(D_f), b(D_u)\}. \end{aligned}$$

The executable prequotation follows exactly the displayed structural clauses. Its relationship to the existing relation is two-way: a related prequote exists exactly when some computational source derivation produces that prequote with budget at most $d$. Conversely, each

supplied derivation produces a related prequote at every sufficient budget. Thus executable normality, original PTS typing, and strong recovery reuse the proved relational construction, without assuming its output as an input witness. After packing the computed blocker telescope, equality of executable packages at any fixed sufficient budget implies exact equality of the original annotated source terms. This injectivity statement does not identify beta-convertible but syntactically different sources. The implementation includes a reducible source $\lambda x : o.(\lambda y : o.y)\, x$: its budget computes to 1, its closed quote computes to a normal term, and the closed decoder recovers the original, still reducible source. A separate computation checks subsequent normalization. These examples test the executable path; the coverage and correctness theorems quantify over all source derivations.

The finite-telescope interface abstracts those blockers in descending order:

$$\begin{aligned}
\mathsf{Code}_{\leq d}(A) &= \Pi b_d : \mathsf{I}_d. \cdots \Pi b_0 : \mathsf{I}_0.A, \\
\ulcorner t \urcorner_d &= \lambda b_d : \mathsf{I}_d. \cdots \lambda b_0 : \mathsf{I}_0.q(t), \\
\mathsf{D}^{\mathrm{tel}}_{d,s} &= \lambda A : \mathcal{U}_s.\lambda q : \mathsf{Code}_{\leq d}(A).q\, \mathsf{id}_d \cdots \mathsf{id}_0.
\end{aligned}$$

For $A : \mathcal{U}_s$, direct rank calculation gives $\mathsf{Code}_{\leq d}(A) : \mathcal{U}_{\max(d+2,s)}$ and a decoder type of rank $\max(d+3, s+1)$. `TelescopeDecoder.family_decoder_typing` checks the actual decoder, with its exact rank expressed by the recursive function `package_rank`. The theorem `fixed_decoder_bounded_quotation` uses the same decoder for every source with budget $d$ and result order $s$.

Closing over the distinguished ground type also has an internal solution. Let

$$\begin{aligned}
K_s &= \Pi o : \mathcal{U}_0.\mathcal{U}_s, \\
C_d(P) &= \Pi o : \mathcal{U}_0.\mathsf{Code}_{\leq d}(P\, o), \\
\mathsf{D}^{\mathrm{cl}}_{d,s} &= \lambda P : K_s.\lambda q : C_d(P).\lambda o : \mathcal{U}_0.(q\, o)\, \mathsf{id}_d \cdots \mathsf{id}_0.
\end{aligned}$$

This is one closed PTS term for each fixed pair $(d, s)$, with type $\Pi P : K_s.C_d(P) \to (\Pi o : \mathcal{U}_0.P\, o)$. For a term-closed source $t : \sigma$ with its single free type variable $o$, put $P_\sigma = \lambda o : \mathcal{U}_0.|\sigma|$ and $Q = \lambda o : \mathcal{U}_0.\ulcorner t \urcorner_d$. The closed representation $Q$ is normal, and the actual decoder application satisfies

$$\mathsf{D}^{\mathrm{cl}}_{d,s}\, P_\sigma\, Q \longrightarrow^*_\beta \lambda o : \mathcal{U}_0.t, \qquad s = \mathrm{ord}(\sigma).$$

`ClosedDecoder.closed_fixed_decoder_bounded_quotation` proves original typing and strong recovery for this closed interface. The ground-type abstraction is inside the decoder; it is not an external evaluation step. Every independent STLC source derivation has a finite budget, so this is an infinite fragment result. The fragment does not include all PTS-rk syntax or the decoder's own polymorphic code, and no self-applicability claim follows.

## 11 Why shallow blocking does not solve dependent quotation

The rank-indexed identities solve a rank-selection problem. They do not by themselves solve dependent conversion. Consider the well-formed context

$$A : \mathcal{U}_0, \quad B : \Pi x : A.\mathcal{U}_0, \quad a : A, \quad f : \Pi x : A.B\, x.$$

The source term $f\,((\lambda x : A.x)\,a)$ can be given type $B\,a$. If its inner redex is replaced by $b_1\,(A \to A)\,(\lambda x : A.x)\,a$, that argument is neutral, so its occurrence in the result type is no longer convertible to $a$. Blocking other applications as well does not restore the lost conversion. This example diagnoses failure of the naive, type-preserving shallow translation; it is not a theorem that every dependent encoding is impossible.

The paper's Figure 2 term-reduction rules do not reduce inside type annotations; PTS-rk's original reduction does. Its normality theorem therefore cannot be imported as the stronger annotation-normality result required here. One cannot avoid the dependent conversion issue by leaving all type expressions untouched: original PTS-rk reduction traverses lambda annotations and products, so a reducible annotation would prevent full normality of the quote. Nor may the typing theorem be replaced by a native Rocq checker judgment, or supplied as a premise of the desired quotation theorem.

A deeper indexed construction needs jointly compatible translations of contexts, types, terms, substitution, and conversion. An explicit interpretation relation or transported representation of equality might help, but the required witnesses must be built inside the original PTS. The canonical-context, hereditary-substitution and distinct-index results above supply this compatibility for metalevel original syntax. Lifting it to a full object-language dependent representation is a separate step. The Tarski operations above supply actual transport programs in a finite normal signature, with typed identity instantiation, normal sort quotations, variable-preserving dependent context extensions and normal local binders. The smart fresh-variable operation solves its local annotation redex, not by itself the global conversion induction or full quotation normality. The checked equality-layer construction above supplies the missing repairs and the full structural quotation theorem. The extensional instantiation functions of Brown–Palsberg suggest a direction; their correctness for $F_\omega$ does not supply this dependent coherence theorem.

# 12 Discussion: comparison with Brown–Palsberg and CPS

## 12.1 What the two self-interpretation results actually say

The relevant comparison is with Matt Brown and Jens Palsberg, *Breaking Through the Normalization Barrier: A Self-Interpreter for $F_\omega$*[4], POPL 2016, especially Sections 3, 5–8 and Figure 4. Their specification combines normal quotation, injectivity and recovery of the represented program. It does not require quotation itself to be an internal operation: their quoter is external, too. Consequently, our use of an executable Rocq function on independent typing-derivation data is not, by itself, a departure from their notion of self-interpretation.

Their principal result has the schematic shape

$$\mathsf{unquote} : \forall \alpha : U.\mathsf{Exp}\,\alpha \to \mathsf{Op}\,\mathsf{Id}\,\alpha, \qquad \mathsf{unquote}\,\ulcorner T \urcorner\,\ulcorner t \urcorner \longrightarrow_\beta^* t.$$

Here $U$ is their universe of represented types, not a PTS-rk sort. The represented type is an ordinary type argument, and the **same fixed term** unquote works for every represented source term. Theorem 7.4 gives reduction to the unreduced source, not merely to its normal form. Our corresponding endpoint is

$$\forall (\vdash t : T)\ \exists j, P, Q. \quad \vdash I_j\,P\,Q : T, \qquad I_j\,P\,Q \longrightarrow_\beta^* t, \qquad I_j, P, Q \text{ normal},$$

[4] https://doi.org/10.1145/2837614.2837623

with $j = (r, k, p, s)$ comprising the tag rank, interface budget, finite numerical equality-layer plan and result rank. The executable construction computes these data from an original derivation. The represented type $P$ and faithful term program $Q$ are ordinary object-language arguments; the interpreter does not take the source type or derivation as a hidden metalevel argument. Nevertheless, $I_j$ is a **family**, not one term with an extra internally supplied natural number. The plan can grow with the derivation even when its source rank bound is fixed. This is a genuine loss of uniformity relative to Brown–Palsberg, not a notational variation on their polymorphic type argument.

There are also substantive gains and qualifications in other directions. We cover the full original dependent PTS, including products whose codomains depend on terms, conversion in types, universe literals and open terms in originally well-formed contexts. All source annotations are retained. Our normality property traverses those annotations; the term-reduction presentation in their Figure 2 does not reduce inside type annotations. Their Definition 6.1 and Theorem 6.3 formulate fidelity modulo beta-equivalence of type annotations. Our additional raw HOAS tag gives literal annotated-syntax fidelity: equality, or beta conversion, of normal faithful programs implies equality of their source terms, even across different derivations and signatures, at a fixed source scope. This is a stronger retention property, not a proof that different derivations of the *same* source produce equal quotations. Such canonical derivation-independence is not claimed. Nor does recovery alone prove syntax fidelity: it only forces source convertibility when codes coincide.

The proof evidence also differs. Brown–Palsberg give mathematical proofs of quotation typing, normality, fidelity and strong unquotation, together with an implemented type checker and experiments. Our corresponding dependent family has kernel-checked Rocq proofs using the unchanged original typing and reduction judgments. That additional verification does not remove the difference in uniformity or supply missing operations on representations. In particular, the single extracted sort example is not performance evidence comparable to a practical implementation of all their operations; our complete finite equality enumeration has no practical cost claim.

### 12.2 Is this a Brown–Palsberg interpreter?

**Yes in the qualified sense of a Brown–Palsberg-style indexed strong unquoter; no as an instance of their original fixed-interpreter result.** The shared core is an external faithful normal quotation and an internally typed evaluator recovering the literal source by beta reduction. Source binders are represented by target binders, operation parameters keep the quote normal, and evaluation instantiates these parameters. Our evaluator actually runs the PHOAS program. It does not project an already evaluated source term stored alongside a syntax tree. The raw tag is used for fidelity, not as a replacement semantic payload.

The fixed-head distinction is essential. The theorem `no_universal_fixed_head_evaluator` in `RankObstruction` rules out one originally typed PTS-rk term whose typed application spines recover every closed originally typed source. Its argument permits arbitrary ordinary type/code arguments, so supplying an internal rank numeral does not evade it. A fixed head bounds the type rank of every typed application spine; there are closed source inhabitants at arbitrarily larger type ranks. This refutes the direct all-rank fixed-head analogue of `unquote`, not arbitrary interfaces that wrap the whole result in externally generated abstractions, nor a metalevel family of heads.

Their self-application experiments also use the same fixed operation on its own representation. Here any originally typed $I_j$ is itself covered by the all-source theorem, hence can be

quoted and recovered by some $I_{j'}$. This is **cross-index self-interpretation**; we have not proved that $j' = j$ is possible or that one fixed index contains its own entire quotation interface. The safe description of the completed result is therefore a *full-dependent, numerical-plan-indexed PHOAS self-interpreter in the strong-unquotation tradition of Brown–Palsberg.* The unqualified phrases "the Brown–Palsberg interpreter for PTS-rk" and "a rank-only self-interpreter" would conceal material differences.

There is a second distinction beyond uniformity. Brown–Palsberg contrast their shallow representation, supporting only unquotation, with a deep representation supporting several interpretations. Their Church-encoded $\mathsf{Exp}$ abstracts over a type operator $F : * \to *$ and four fixed handlers: abstraction, application, type abstraction and type application. Figure 4's $\mathsf{foldExp}$ replaces these parameters to obtain unquotation, an abstraction test, size, a normal-form test, and CPS. Our full-dependent representation abstracts over a finite Tarski signature and checked equality witnesses. It supports typed instantiation, but the completed all-source theorem constructs only its identity interpretation. Raw-tag folds and the earlier simply typed PHOAS algebra are additional infrastructure; they do not already give these five interpretations of the **full dependent** program. Thus HOAS binders, executable unquotation and a raw syntax tag are not by themselves the reusable deep fold result of the paper.

### 12.3 The CPS benchmark in the paper

Brown–Palsberg's Figure 4 defines an answer-type-polymorphic continuation operator and interprets the same representation under it:

$$\mathsf{Ct}\, A = \forall R : *.(A \to R) \to R, \qquad \mathsf{CPS} = \mathsf{Op}\, \mathsf{Ct},$$

$$\mathsf{cps} : \forall \alpha : U.\mathsf{Exp}\, \alpha \to \mathsf{CPS}\, \alpha.$$

The four continuation handlers recursively transform the represented program by a fold. Their displayed transformation is call-by-name; they also report an implemented call-by-value variant. Importantly, its output is a **CPS program**, not necessarily a new $\mathsf{Exp}$ syntax code. An operation returning a new faithful normal quotation of that program would be a further, stronger requirement. They explicitly leave the semantic correctness proof of CPS open, while reporting its implementation, type checking, tests and self-applicability. We should neither attribute a proved CPS simulation theorem to them nor claim that merely writing $\lambda k.k\, t$ meets their implemented transformation benchmark.

For PTS-rk we must separate three questions: whether continuation types and transformations can be expressed at larger ranks; whether the **existing** full-dependent code can be instantiated with CPS handlers; and whether such an internal transformation has type preservation and semantic correctness for every original source. Only the first admits the immediate positive calculations below. The latter two remain new obligations, not consequences of the interpreter theorem.

### 12.4 Continuations fit at higher ranks

The following calculations are paper-level derivations from the original PTS rules, not newly added Rocq CPS theorems. If $A : \mathcal{U}_a$ and $R : \mathcal{U}_b$, define

$$\mathsf{Cont}_R(A) = (A \to R) \to R, \qquad h(a,b) = \max(a+2, b+1).$$

The two product formations give

$$A \to R : \mathcal{U}_{\max(a+1,b)}, \qquad \mathsf{Cont}_R(A) : \mathcal{U}_{h(a,b)}.$$

Answer-type polymorphism at a fixed external answer rank is also legal:

$$\mathsf{Ct}_b(A) = \Pi R : \mathcal{U}_b.(A \to R) \to R : \mathcal{U}_{h(a,b)}.$$

Indeed $\mathcal{U}_b : \mathcal{U}_0$, and the outer product has rank $\max(1, h(a,b)) = h(a,b)$. The computation is finite and effective. However, $h(a,b) > a$: this is not an endofamily $\mathcal{U}_a \to \mathcal{U}_a$ of the kind one would obtain by literally copying $F : * \to *$. There is no cumulativity to repair a mismatch silently. Instantiating the answer binder with $A$ itself requires $A : \mathcal{U}_b$, hence the appropriate matching rank, not just an inequality between ranks. Finite rank-indexed continuation interfaces are plausible; one same-rank $\mathsf{Ct}$ covering the existing interface is not a drop-in instance.

A concrete positive test is the simply typed fragment. Fix ground and answer types $O : \mathcal{U}_a$, $R : \mathcal{U}_b$, and define value and computation types structurally by

$$V(o) = O, \qquad C(A) = \mathsf{Cont}_R(V(A)), \qquad V(A \to B) = C(A) \to C(B).$$

Their ranks are computed by

$$v(o) = a, \qquad c(A) = \max(v(A)+2, b+1), \qquad v(A \to B) = \max(c(A)+1, c(B)).$$

With $x : A$ translated to $x : C(A)$, a call-by-name translation is

$$\begin{aligned} \mathcal{C}(x) &= \lambda k : V(A) \to R.\, x\, k, \\ \mathcal{C}(\lambda x : A.t) &= \lambda k : V(A \to B) \to R.\, k\, (\lambda x : C(A).\mathcal{C}(t)), \\ \mathcal{C}(f\, u) &= \lambda k : V(B) \to R.\, \mathcal{C}(f)\, (\lambda g : C(A) \to C(B).\, g\, \mathcal{C}(u)\, k). \end{aligned}$$

The variable clause explicitly introduces the continuation lambda; it does not invoke an eta-conversion rule. A structural typing induction assigns each right-hand side $C$ of the source type, using precisely the displayed finite ranks. These clauses have the HOAS-friendly form needed for abstraction and application handlers. This is evidence that predicative rank growth and HOAS are not general obstacles to CPS. It is not a new implementation in `PHOASAlgebra`: its current rank-preserving family contracts would need a reindexed interface, and the full dependent case has additional obligations absent from simple types.

Even this particular translation needs more than a bound on the source type's rank. Take $a = b = 0$, $A_0 = o$ and $A_{n+1} = o \to A_n$. Every $A_n$ with $n \geq 1$ has source rank 1, whereas the displayed recurrence gives $v(A_n) = 2n+1$ and $c(A_n) = 2n+3$. This follows by induction from $v(A_1) = 3$ and $v(A_{n+1}) = \max(3, v(A_n)+2)$. Thus a finite target budget can be computed from the type structure, but this recipe is not rank-only uniform. This is a limitation of the displayed translation, not a lower bound for every possible CPS encoding.

Even for the full interpreter there is an immediate but much weaker operation. With $D = I_j\, P\, Q : T$ and $T : \mathcal{U}_a$, ordinary original typing gives

$$\lambda k : T \to R.\, k\, D : \mathsf{Cont}_R(T), \qquad \lambda k : T \to R.\, k\, D \longrightarrow_\beta^* \lambda k : T \to R.\, k\, t.$$

The represented type can supply the corresponding decoded type expression in the generic interface. This is a return wrapper around direct execution, not a transformation of internal applications. The all-source quotation theorem can subsequently quote such a typed wrapper at a new index, but that still does not turn it into a recursive CPS compiler.

### 12.5 Why CPS is not a substitution into the current full signature

For a fixed signature $\mathcal{S}$, a raw term program has the shape $q : \Pi\mathcal{S}.\mathsf{El}_s(P\,\mathcal{S})$. Any originally typed filling $\rho$ of **every** signature field gives a typed interpretation $q\,\rho$ by substitution. This is the correct entry point for a new fold, but the hypothesis is substantial.

First, the product fields carry the law

$$\mathsf{Eq}_{c+1}\big(\mathcal{U}_c, \Pi x : \mathsf{El}_a A.\mathsf{El}_b(Bx), \mathsf{El}_c(p_{a,b}\, A\, B)\big), \qquad c = \max(a+1, b).$$

If decoding a product code is changed into a continuation-wrapped function type, while the left endpoint remains the ordinary dependent product of the decoded constituents, these endpoints are no longer definitionally the same. The identity filling's reflexivity witness no longer checks. Moreover, each decoder field has a fixed target sort $\mathcal{U}_r$, whereas the naive continuation replacement increases that sort. Repairing rank budgets alone does not repair the product law. Conversely, merely noting non-convertibility does **not** prove that its internal equality type is uninhabited: that stronger claim would require a separate argument.

Second, checked equality layers were accepted by testing convertibility *after the identity filling*. The decision procedure did not test validity under every possible algebra. A CPS instance would need actual originally typed witnesses for all these instantiated fields, including their dependent local telescopes. The erasure property of an identity realizer says nothing about a continuation interpretation of that same field. Supplying an arbitrary equality axiom, adding eta conversion, or postulating a free theorem would change the problem.

Third, in a dependent application $f : \Pi x : A.B(x)$, $u : A$, the result type is $B(u)$. Replacing $u$ by a computation changes not just evaluation order but the argument of a type family. A translation must explain how its value/computation distinction interacts with type-level substitution and source conversion. The Rocq `QuotationTree` and cache data are external construction metadata, not data that an internal PTS program can automatically recover from $P, Q$. Their availability to our external quoter is not an implementation of this missing internal operation.

One small counterexample shows why a *naive call-by-value* replacement cannot preserve all source conversions by beta conversion alone. In the simply typed source context $g : O \to O$, $v : O$, $y : O$,

$$(\lambda x : O.y)\,(g\, v) \longrightarrow_\beta^* y.$$

Under the standard fixed-answer call-by-value translation, write $\widehat{g} : O \to \mathsf{Cont}_R(O)$ for the translated variable. After administrative beta steps the two computation terms are

$$\lambda k : O \to R.\, \widehat{g}\, v\, (\lambda z : O.k\, y), \qquad \lambda k : O \to R.\, k\, y.$$

For neutral variables and normal annotations these are distinct beta-normal terms, hence not beta-convertible. They may be identified by a suitable observational or parametric semantics, but not by the original conversion judgment. Source dependent types may use exactly such unrestricted beta conversions. This refutes the direct conversion-preserving CBV recipe, not type-preserving CPS with explicit transports or a different indexing discipline. It also does not refute call-by-name CPS; the paper's displayed transformation is call-by-name, and deserves a separate investigation.

Related positive results do not discharge these obligations automatically. Barthe, Hatcliff and Sørensen (1999)[5] treat dependent CPS, including routes for annotated pure type systems. Their Definition 11 requires a distinguished logical sort with $(\mathsf{Prop}, \mathsf{Prop}, \mathsf{Prop})$ among the product rules. PTS-rk has no such sort: $\max(a+1, a) = a+1 \neq a$ for every $a$. The candidate $\mathcal{U}_0$ also violates their prohibition on axioms into $\mathsf{Prop}$, since every sort has type $\mathcal{U}_0$. Thus the logical-PTS theorem cannot simply be instantiated here; this is a failure of its hypotheses, not a no-CPS theorem. Bowman, Cong, Rioux and Ahmed (2018)[6] prove type-preserving call-by-name and call-by-value translations for dependent $\Pi$ and $\Sigma$ types using answer-type polymorphism. Their target adds a typing rule and an equivalence rule justified through a parametric model in extensional Calculus of Constructions. Importing those rules into our unchanged beta-only PTS is not authorized by our interpreter theorem. Explicit internal evidence would have to replace any use of the additional rules.

### 12.6 Feasibility verdict and a concrete continuation of the project

There is **no established general obstruction to indexed dependent CPS**, but there is also **no full-dependent CPS fold on the present internal representation**. Higher-rank continuation types and the simple-type translation are positive evidence. The unchanged current interface rejects the literal Brown–Palsberg continuation substitution, for both rank and dependent-law reasons. Full-source normalization or strong unquotation alone does not construct an alternative algebra.

Two developments would settle different versions of the question.

1. An external, derivation-directed CPS translation, preferably starting with call-by-name, must translate contexts, types, terms and all six original typing rules, preserve the required substitution/conversion relations, and compute a finite target rank budget. If it supplies an original target derivation, our existing quoter can turn that CPS program into a normal faithful PHOAS code at a new computed index. This would establish a typed source-to-code CPS pipeline, **not** an internal operation on $P, Q$.

2. A Brown–Palsberg-style internal operation must instead construct a continuation interpretation of the signature, with actual witnesses for its laws, or redesign the representation to support one. A potentially useful redesign retains both a direct value for dependent type indices and a continuation computation, with a proved relation between them. It must work under binders and account for each equality repair; merely pairing an arbitrary computation with a direct value does not establish that relation. Its output index bounds must depend on the allowed input interface and any declared budgets, rather than secretly receiving the source derivation.

[5] `https://www.cs.ru.nl/~freek/courses/tt-2017/papers/10.1023-A-1010000206149.pdf`
[6] `https://www.williamjbowman.com/resources/cps-sigma.pdf`

For the second route the first decisive test is a typed continuation product/application handler satisfying the dependent product law and one nontrivial contextual equality layer. Only after that test should we claim that the full representation supports the paper's deep CPS interpretation. A simulation or logical-relations theorem is a further correctness goal. None of these CPS obligations is included in the current closed Rocq assumption reports. The completed strong unquoter remains useful infrastructure for them, but is not their proof.

## 13 Formalization status and next obligations

The separate library is `PTSrk.SelfInterpreter`, physically in `theories/self_interpreter`. It imports the original kernel and existing normalization infrastructure; neither arithmetic interpretation nor either set-model library is a dependency. The reference ZFC development is unchanged.

The `ChurchSyntax` modules implement raw HOAS, fidelity and folds. `BrownPalsberg` proves the single-family obstruction and indexed type interpretation. The three `TypedHOAS` modules provide source-indexed prequotation, syntax fidelity and the actual typed identity interpretation. The new `PHOASAlgebra`, `PHOASFaithful`, `PHOASTelescope`, `PHOASRecovery`, `PHOASPackage` and `PHOASDecoder` modules close that construction over a finite rank-indexed palette and supply an actual strongly recovering closed decoder for each budget and simple source type. `DependentPHOAS` constructs genuinely dependent local handlers and a candidate universe decoder; `PHOASConversion` proves the limited shared-map coherence obstruction. `OpenCanonical`, `CanonicalSubstitution`, `CanonicalContexts` and `CanonicalTypeIndex` provide the full-PTS open-context canonical index algebra, with hereditary substitution equations and bidirectional original typing transport, separately from an internal evaluator. `IndexedEquality`, `TarskiProducts`, `TarskiUniverses`, `TarskiCasts`, `TarskiIdentity` and `ClosedTarskiCasts` provide the local object-language code/transport interface and actual identity realization. `TarskiLawFamily`, `TarskiPalette` and `TarskiSignature` assemble its finite normal context and typed identity substitution; `TarskiOperations` proves the applied-operation typings and substitution recovery. `TarskiSorts` supplies normal closed sort quotations with directly typed identity application. `TarskiContext` lifts the signature and its typed identity substitution through dependent binders. `TarskiBinders` supplies smart local Abs/App, fresh-variable coherence and an originally typed normal dependent binder with local input fidelity. `ScopedTarski` extends these operations to arbitrary dependent scopes. `TarskiBeta` and `TarskiBetaSignature` add a finite internal beta-law block with constructive typed identity realization. `FullSource`, `FullConversion` and `CanonicalSource` provide exact computational source derivations and certified forward conversion joins in well-formed contexts, preserving literal source syntax rather than replacing it by normal forms. `SourceSubstitution` computes derivations under substitution and of literal beta contracta. `IndexedEqualityLevels` supplies predicative reversal and joins; `DependentEquality` and `TarskiGraphBeta` connect synchronized dependent evidence to the actual finite beta fields. `BinderCongruence` constructs the common-map binder and product-code cases without arbitrary pointwise-to-function equality. `ContextualTelescope`, `ContextualBeta`, `TelescopeBetaFamily` and `UniformContextualBeta` construct the contextual cast-beta family and its independent identity program. `UniformTelescope` and `UniformTelescopeRecovery` supply normal domain parameters and actual concrete-telescope instantiation. `ContextualBudgets` and `TelescopeBetaSignature` give its finite syntax-free interface; `ContextualBetaFamily` and `ContextualBetaSignature` retain the explicit one-binder specialization. `RealizedSignature` separately packages any finite family whose closed realizations have already been independently typed. `IndexedCoherence` constructs equality

composition and nondependent congruence. `TarskiRoundtrip` checks the precise neutral-cast cancellation failure, while `EtaBoundary` checks the limited pointwise-versus-beta regression for dependent indices. The earlier `RankedQuotation` and `ExecutableQuotation` modules retain the independent shallow STLC quotation, with actual closed decoders in `TelescopeDecoder` and `ClosedDecoder`. `RankedNormalization` connects full PTS normalization to canonical HOAS codes; `LiteralObstruction` distinguishes redex/type-rank indices from the needed literal ceiling. `WeakFullInterpretation` supplies the full closed weak semantic-payload construction, including normality and exact source fidelity. None of these modules assumes the manuscript's unformalized elementary compiler. Public endpoints and limits are listed in the development README and checked by `Audit`.

The development uses the pinned Rocq 9.2.0, Stdlib 9.1.0, Equations 1.3.2+9.2, OCaml 4.14.2, and Dune 3.23.1 environment. Fresh assumption reports and recursive kernel checks are compared with a pristine upstream build. No new axioms, admitted proofs, unsafe casts, modified PTS rules, or disabled checks are part of this development. Verification commands and failures are recorded separately from theorem claims.

The all-source numerical-plan construction establishes original typing, full normality, annotated-syntax injectivity, strong recovery and full coverage in well-formed original contexts. Its final implementation is `FullDependentInterpreter`, based on `RecursiveQuotation`, `ApplicationQuotation`, `LocalCodeAlignment` and `IndexedProgramInterpreter`. It is independent of the weak semantic-payload construction and of the manuscript's unformalized elementary compiler.

The remaining research programme concerns stronger interfaces and costs:

1. Formalize the non-uniform elementary compiler, including the corrected modulus construction, and instantiate it with fixed-budget normalization and elementary serialization. Prove original numeral-program typings and beta equations; a native arithmetic computation is not that proof.
2. Construct actual typed HOAS-to-numeric adapters and normal-form reification with sufficient rank and literal indices. The latter must return an inhabitant of the represented dependent type, not just another code. This would strengthen the full-PTS weak construction by eliminating its externally supplied normal semantic payload.
3. Determine whether the numerical plan can be replaced by a rank-only index with one fixed interpreter covering each whole rank-bounded fragment. The current proof gives a finite plan per derivation, not a rank-only bound on plan length or syntax enumeration.
4. Replace exhaustive checked equality layers by a smaller uniform coherence interface, preserving original annotations and beta-only computation. Prove the relevant quotation and evaluator operation costs.
5. Study extracted execution beyond leaf regressions, and distinguish its proof erasure from direct evaluation of dependent records in Rocq. Canonical derivation-independent quotation and a pure raw-HOAS decoder are additional specifications, not consequences of source coverage.

Thus a full dependent numerical-plan-indexed PHOAS interpreter is constructed. Neither a rank-only family nor an all-rank single program is claimed. The original complexity argument remains a separate conditional route to numeric self-reduction; it is not used as an assumed typed reifier.